

Computernetze und verteilte Systeme

Zusammenfassung



TECHNISCHE
UNIVERSITÄT
DARMSTADT

Author: Bassem Zouari

Version: v1.0

Semester: SoSe 2025

Stand: 14. Mai 2026

Fachbereich: Informatik

Inhaltsverzeichnis

1	A Quick Tour through Networking	2
1.1	Distributed Systems (DS)	2
1.2	Netzwerke im Überblick	3
1.3	Kommunikation zwischen zwei Geräte	4
1.4	Netzwerkstruktur & Vermittlungsverfahren	8
1.5	Multiplexing & Shared Resources	11
1.6	Netzwerkorganisation & Abstraktion	13
1.7	OSI-Modell	15
2	Routing	20
2.1	Introduction	20
2.2	Nicht-adaptive Routing Algorithmen	26
2.3	Adaptive Routing Algorithms	27
2.4	Distance Vector Routing (DVR)	30
2.5	Link-State Routing (LSR)	35
2.6	Hierarchical Routing	36
2.7	Reflektionen über Internet-Routing	39
2.8	Mobile Routing - Herausforderungen & Lösungen	41
2.9	Overlay Routing	43
3	Internetworking + DNS	45
3.1	Internet Protocol (IP)	45
3.2	Domain Name Service (DNS)	49
4	Transport Layer	54
4.1	Connection Control	54
4.2	Flow Control	63
4.2.1	Alternating Bit Protocol (ABP)	65
4.3	Error Control	67
4.3.1	Automatic Repeat reQuest (ARQ)	68
4.4	Congestion Control	70
4.5	Internet UDP	79

1 A Quick Tour through Networking

1.1 Distributed Systems (DS)

Definition

Ein verteiltes System ist ein Zusammenschluss mehrere Computer, die über ein Netzwerk miteinander verbunden sind und **als ein einziges System** erscheinen und arbeiten

Zitat zur Veranschaulichung

Laslie Lamport: „Ein verteiltes System ist eines, bei dem der Ausfall eines Computers, von dessen Existenz man nicht einmal wusste, den eigenen Computer unbenutzbar machen kann“

Das zeigt: **Ein Fehler irgendwo kann das ganze System betreffen**

Eigenschaften eines verteilten Systems

- Besteht aus **mehreren Computern** (oft heterogen)
- Verbindet diese über ein **Computernetzwerk**
- Die Computer **kooperieren eng**, um **gemeinsam eine Aufgabe oder einen Dienst zu erfüllen**
- Das System funktioniert **nur über das Netzwerk**
- Das System wirkt **nach außen wie ein einziges** („System of systems“)

Herausforderungen/Probleme

- Verzögerungen im Netzwerk (Latenz)
- Ausfall einzelner Komponenten
- Datenkonsistenz und Synchronisation
- Fehlende zentrale Kontrolle

Beispiel für verteilte Systeme

- **Internet**
- **IPTV** (Fernsehen über Internet)
- **VoIP** (z.B. Skype, Zoom)
- **Peer-to-Peer Netzwerke** (z.B. BitTorrent)
- **Online-Spiele**
- **Metaverse / Virtuelle Welten**

Merksatz

„Kein verteiltes System ohne Computernetzwerk“

1.2 Netzwerke im Überblick

Allgemeines

- Netzwerke sind ein **junges, schnell wachsendes Fachgebiet**
- Gleichzeitig aber auch alltäglich geworden: eine „**Commodity**“ bzw. „**Utility**“ (wie Strom oder Wasser)

Ständige Veränderung (Constant Flux)

- **Rasante technologische Entwicklung:**
 - **Kehrrnetzwerke:** Von T1 (1,544 Mbps) → zu 100-Gbps-Switches (und mehr)
 - **Heimanwender:** 1,2 kbps Modem → ADSL2 + (24 Mbps) → VDSL2-VPlus (300 Mbps) → Glasfaser (n x 1 Gbps)
- **Wandel in der Infrastruktur und Kommunikation:**
 - Geräte: Workstations → PCs → Smartphones → Sensoren → Kommunikation & Messung kombiniert
 - Betriebssysteme: Unix → Linux/BSD → Windows, macOS → iOS, Android, TinyOS → **Jedes Gerät hat heute ein OS**

Vergleich: Programmiersprachen vs. Netzwerktechnik

Bereich	Programmiersprachen	Netzwerke / Verteilte Systeme (CN & DS)
Theorie	Klar definierte Grundlage	Viele verschiedene Modelle
Struktur	Gut organisierte Forschung	Forschung eher unstrukturiert
Reife	Mehrere Phasen der Reifung durchlaufen	Teilweise noch in den Kinderschuhen?
Realität	Softwarequalität oft sehr schlecht	Große funktionierende Systeme existieren

Merksätze

- „Networking ist jung, aber überall“
- „Netze verändern sich ständig - Geräte, Dienste, Kommunikation, Betriebssysteme“

1.3 Kommunikation zwischen zwei Geräte

Beispiel 1: World Wide Web

Was passiert, wenn du `www.tu-darmstadt.de` in die Adresszeile eingibst und Enter drückst?

1. **Namensauflösung (DNS):** Domainname → IP-Adresse
2. **Verbindungsaufbau:** z.B. über TCP
3. **Anfrage (Request):** Browser sendet HTTP-Request
4. **Antwort (Response):** Webserver liefert die Webseite zurück
5. **Darstellung:** Browser zeigt die Seite an

→ **Übertragung einer Datei oder von Datenpaketen**

Beispiel 2: Telefonanruf

Was passiert, wenn du jemanden anrufst?

1. **Teilnehmernummer finden & Verbindungsaufbau**
2. **Sprachdaten in Echtzeit übertragen** (z.B. über VoIP oder klassische Telefonleitung)
3. **Beidseitiger, synchroner Austausch**

→ **Kontinuierlicher Informationsfluss - muss rechtzeitig ankommen!**

Vergleich Webseite vs. Telefonat

Merkmal	Webseite (HTTP)	Telefonat (VoIP / klassisch)
Datenart	Statische Datenmenge (z.B. HTML-Datei)	Kontinuierlicher Strom (Sprache)
Zeitkritisch?	Nein (Daten können gepuffert werden)	Ja (muss in Echtzeit ankommen)
Interaktivität	Einseitig oder zeitverzögert	Echtzeit, bidirektional

Was ist eigentlich „Daten“ und „Information“?

Begriff	Erklärung
Daten	Rohwerte, z.B. Zahlen, Zeichen - ohne Kontext noch bedeutungslos
Information	Wenn Daten in einem Kontext interpretiert und verstanden werden

Beispiel:

- „42“ = **Daten**
- „42 Grad Celsius Körpertemperatur“ = **Information**

Einfachste Kommunikation: Direkte physikalische Verbindung

Beispiele:

- **WWW:** Client (Browser) direkt per Kabel mit Server verbunden
 - Client sendet Anfrage, Server liefert Daten
- **Telefonie:** Zwei Telefone direkt per Kabel verbunden

Wichtige Begriffe: Information - Daten - Signale

Abstrakte Ebene	Reale Umsetzung
Information	Bedeutung, Inhalt
Daten	Darstellung der Information (z.B. als Bits)
Signale	Physikalische Repräsentation von Daten (z.B. elektrische Spannung, Schall)

Was ist ein Signal?

Ein **Signal** ist die Darstellung von Daten durch charakteristische Änderungen eines physikalischen Werts (über Raum oder Zeit)

- **Materiell:** Buchstaben auf Papier
- **Immateriell:** Schallwellen beim Sprechen
- **Elektrisch:** Spannung oder Strom im Draht

Bits und Signale

Was soll übertragen werden?	Daten (als Bits)
Was kann physisch übertragen werden?	Signale (z.B. Spannung)

Wir brauchen:

- **Kodierung:** Bits → Signale (Sender)
- **Dekodierung:** Signale → Bits (Empfänger)

Einfaches Beispiel (nicht praxisnah):

- Bit 1 = Strom vorhanden
- Bit 0 = Kein Strom

Mekrsätze

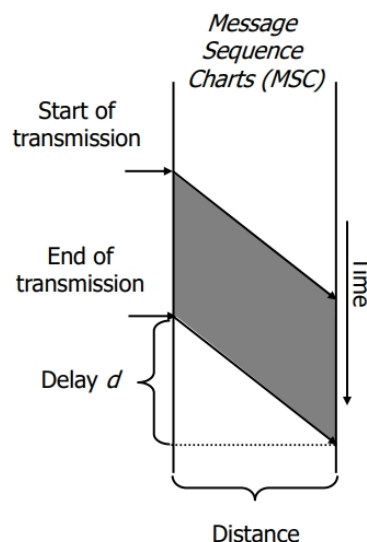
- „Daten werden zu Signalen, damit Maschinen miteinander sprechen können“
- „Ein Kabel speichert Daten, indem es sie unterwegs hält“
- „Nur mit Signalen erreichen Bits ihr Ziel“

Technische Eigenschaften der Übertragung

Begriff	Erklärung
Verzögerung (Delay d)	Wie lange braucht ein Signal
Signalausbreitung (v)	Geschwindigkeit der Signalverbreitung (z.B. Lichtgeschwindigkeit c , in Kupfer: ca. $2/3 \cdot c$)
Datenrate (r)	Wie viele Bits pro Sekunde können gesendet werden?
Fehlerrate	Wie viele Bits kommen fehlerhaft an? Welche Fehlerbilder treten auf?

Wie kann ein Kabel Daten „speichern“?

- Während der ersten d Sekunden ist das erste Bit **unterwegs**, aber noch **nicht angekommen**
- In dieser Zeit hat der Sender bereits $d \cdot r$ Bits gesendet - diese Bits **befinden sich im Kabel** → „gespeichert“ in der **Leitung**



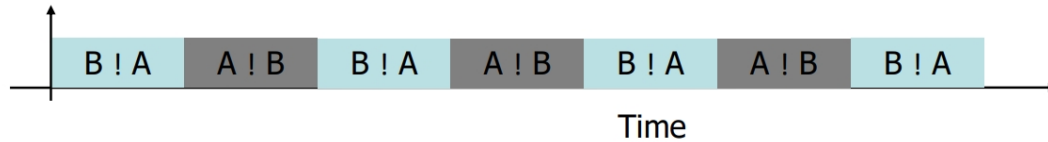
Zwei-Wege-Kommunikation: Übertragungsarten

Art	Beschreibung	Beispiel
Simplex	Nur eine Seite sendet	Rodio
Halbduplex	Sender und Empfänger wechseln sich ab	Walkie-Talkie, Gespräch
Vollduplex	Beide Seiten können gleichzeitig senden	Telefonat, Webkommunikation

Halbduplex: Wechselnde Kommunikation

- **Problem:** Beide senden gleichzeitig → Signale stören sich
- **Frage:** Wie erkennt ein Teilnehmer, wann der andere fertig ist?

Lösungen:



1. Time Division Duplex (TDD)

→ Feste Zeitslots, synchronisiert

Beispiel: A darf 10 ms senden, dann B für 10 ms usw.

2. On-Demand / dynamisch:

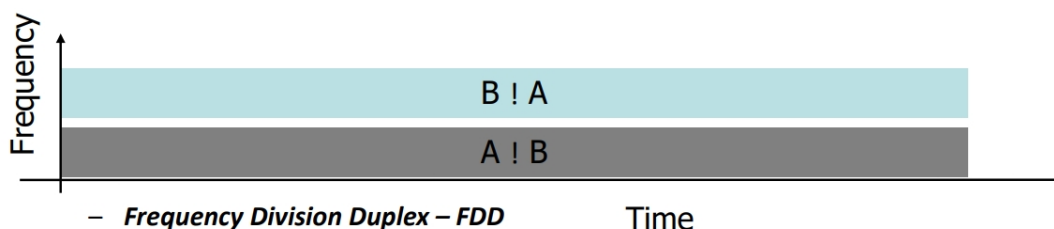
- Sender kündigt Sendedauer vorher an
- Oder: schickt spezielles „Ende“-Signal

Herausforderungen:

- Max. Länge muss definiert sein
- „Ende“-Signal darf **nicht Teil der Nutzerdaten** sein
- **Bit-Synchronisation** nötig

Vollduplex - Gleichzeitiges Senden & Empfangen

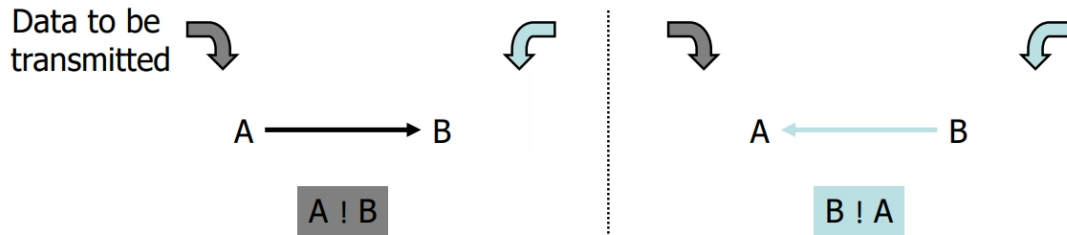
- **Kabelgebunden:**
 - Zwei separate Kabel (je Richtung) → zuverlässig, aber aufwändig (Kosten, Platz, Wartung)
 - **Ein Kabel?** Möglich auf kurze Distanz (z.B. mit Signalverarbeitung, z.B. Echo-Unterdrückung)
- **Drahtlos / Funk / Kabel-TV:**
 - **Frequenz Division Duplex (FDD):**
Unterschiedliche Frequenzen für Hin- und Rückrichtung
→ **keine Kollision** durch Frequenztrennung



Voll-Duplex mit TDD - Geht das überhaupt?

Klingt widersprüchlich, aber:

- Wenn **Übertragungsmedium mindestens doppelt so schnell** ist wie die benötigte Datenrate, dann:
 - Teilnehmer A speichert Daten zwischen (z.B. als Datei oder in einem Speicherchip), während B sendet
 - Danach sendet A **zwischengespeicherte + neue** Daten in kürzerer Zeit
 - So wirkt es wie **gleichzeitiges Senden**

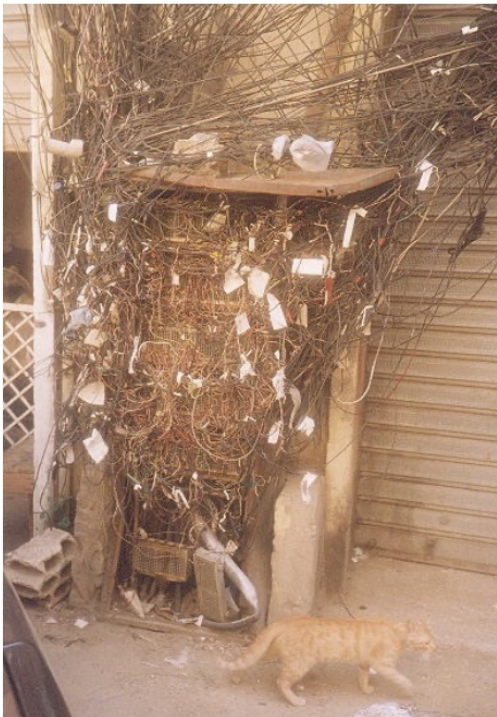


- **Funktioniert bei digitalen Signalen**
- **Nicht geeignet für analoge Signale** (z.B. klassische analoge Telefonie)
 - Da keine exakte Zwischenspeicherung möglich ist

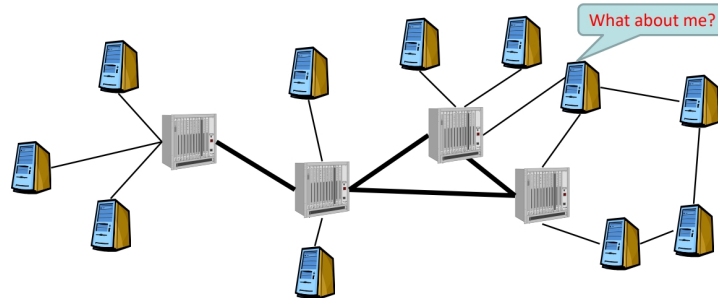
1.4 Netzwerkstruktur & Vermittlungsverfahren

Warum nicht jedes Gerät direkt verbinden?

- Direkte Verbindung **jedes Geräts mit jedem anderen** ist **nicht skalierbar**
 - Bei n Geräten: $n(n-1)/2$ Verbindungen nötig (explodiert schnell!)



- Lösung: **Struktur im Netzwerk** einführen



Netzwerkstruktur

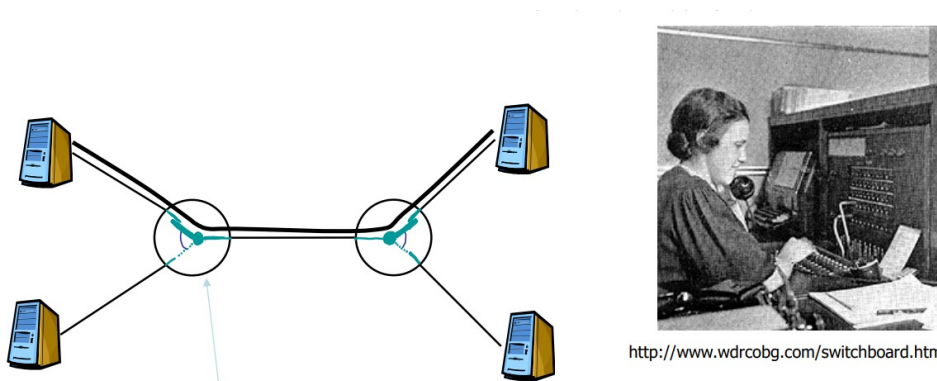
Zwei Arten von Komponenten:

- **Endsysteme:** Benutzergeräte (PCs, Smartphones, etc.)
- **Vermittlungselemente:** Router, Switches

Vermittlungsarten (Switching)

1. Leitungsvermittlung (Circuit Switching)

- Eine **dedizierte Verbindung** (Schaltkreis) wird **vorab** zwischen zwei Endsystemen aufgebaut
→ Wie klassisches Telefonnetz (analog: „Fräulein vom Amt“)
- **Verbindung bleibt exklusiv** für die Dauer der Kommunikation bestehen
- **Vorteile:**
 - Feste Ressourcen → **garantierte Übertragung**
 - **Keine Verzögerung durch Warteschlangen** (kein Queuing)
- **Nachteile:**
 - Ressourcen **blockiert**, auch bei Pausen
 - Verbindung muss **vor dem Datentransfer aufgebaut** werden
 - Wenig effizient bei kurzen Nachrichten oder vielen Teilnehmern



manual ~1880 → electromechanical (pulse) ~1910 → electric → digital (switch) → fully digital

2. Paketvermittlung (Packet Switching)

- Daten werden in **Pakete** aufgeteilt (jeweils mit Zieladresse)
- Pakete werden **einzel und unabhängig** durch das Netz geschickt
- Kein fester Pfad nötig → **dynamisches** Routing möglich

Eigenschaften:

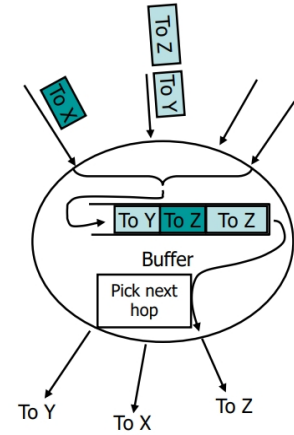
- Jeder **Switch** muss:
 - a) Paket empfangen
 - b) Zwischenspeichern (Buffering)
 - c) Zieladresse auslesen
 - d) Nächsten Weiterleitungspfad bestimmen
 - e) Paket weiterleiten („store-and-forward“)

Vorteile:

- **Effiziente Ressourcennutzung**, da keine exklusive Leitung
- Flexibel & robust bei Ausfällen oder Störungen

Herausforderungen:

- Start/End eines Pakets erkennen
- Adressing und Routing
- Pufferüberläufe & Status (Congestion)
- Höherer **Verwaltungsaufwand** durch Header & Steuerinformationen



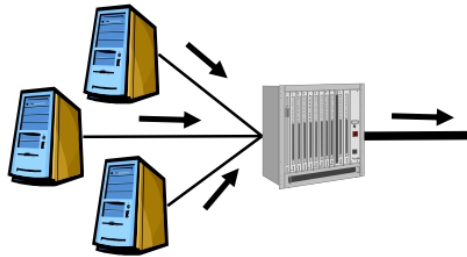
Merksätze

- „Direkte Verbindungen skalieren nicht - Vermittlung ist nötig“
- „Leitungsvermittlung = reserviert & einfach, aber unflexibel“
- „Paketvermittlung = flexibel & effizient, aber komplexer“

1.5 Multiplexing & Shared Resources

Multiplexing Definition:

- Organisation der Übertragung mehrere Datenströme über eine einzige physische Verbindung
→ Verschiedene Sender teilen sich ein Übertragungsmedium
- Das Ziel ist es, **eine Leitung effizient zu nutzen**, statt für jede Verbindung eine eigene Leitung bereitzustellen



Wie funktioniert Multiplexing konkret?

Beispiel: Zwei PCs senden gleichzeitig an denselben Empfänger

- Wenn beide gleichzeitig auf derselben Leitung senden, **würden sich ihre Signale überlappen** → **Signal-Kollision!**
- Multiplexing sorgt dafür, dass diese Übertragungen **geordnet** und **kollisionsfrei** ablaufen - entweder in der Zeit, über verschiedene Frequenzen, oder durch andere Techniken

Multiplexing-Methoden

Methode	Beschreibung	Beispiel
TDM (Time Division Multiplexing)	Jede Verbindung bekommt eine feste Zeit-spanne (Zeitschlitz) zur Übertragung	Telefonnetz, Ethernet
FDM (Frequency Division Multiplexing)	Daten werden gleichzeitig, aber auf unterschiedlichen Frequenzen gesendet	UKW-Radio, Kabelfernsehen
WDM (Wavelength Division Multiplexing)	Wie FDM, aber in der optischen Kommunikation (Glasfaser)	Glasfasernetze
CDM (Code Division Multiplexing)	Jeder Sender bekommt einen eindeutigen Code	Mobilfunk (3G/UMTS)
SDM (Space Division Multiplexing)	Verschiedene Signale laufen über räumlich getrennte Kanäle, z.B. in Mobilfunkzellen	Mobilfunknetze (4G/5G)

Demultiplexing

Gegenstück zu Multiplexing: Trennt am Empfänger die einzelnen Datenströme wieder auf

Upward vs. Downward Multiplexing

Upward	Viele logische Verbindungen teilen sich eine physische Verbindung	Mehrere Apps auf deinem Handy nutzen gleichzeitig das WLAN
Downward	Eine logische Verbindung nutzt mehrere physische Verbindungen	Download-Boosting durch mehrere Leitungen

Shared Resources

- Multiplexing regelt den **Zugriff auf gemeinsam genutzte Ressourcen** - z.B. ein drahtloses Medium
- **Beispiele für geteilte Ressourcen:**
 - Funkfrequenzen (Luft als Übertragungsmedium)
 - Gemeinsamer Draht (Kupferkabel)
 - Ein Netzwerk-Switch oder Routerport

Das Problem der Mehrfachzugriffe (Multiple Access)

- **Broadcast-Medium** (z.B. Luft, Funk, geteiltes Kabel) → Daten erreichen alle Teilnehmer
- **Nur ein Sender zur gleichen Zeit erlaubt** → sonst: Kollisionen
- Lösung: **Medium Access Control (MAC)**

Zugriffsverfahren (Medium Access Control / MAC)

Verfahren	Beispiel	Erklärung
Zentralisiert (Lehrer-Schüler-Prinzip)	Unterricht	Nur der Lehrer gibt Schülern das Wort (Zugang zum Medium)
Dezentralisiert (Party-Prinzip)	Gesprächsrunde	Jeder versucht zu sprechen, wartet auf Pause - Kollisionen möglich, dann Rückzug & Wiederholung

Forwarding und Next Hop Selection

Was ist Forwarding?

- Forwarding ist der **Prozess, bei dem ein Router entscheidet, wohin ein Datenpaket als Nächstes geschickt werden soll**, damit es sein Ziel erreicht
- Es geht **nicht** darum, den ganzen Weg zu planen, sondern nur den **nächsten Schritt** - den sogenannten **Next Hop**

1.6 Netzwerkorganisation & Abstraktion

Ziel: Wie können viele, verschiedene („alien“) Computer effizient kommunizieren?

Wichtigstes Konzept: Abstraktion

- **Abstraktion** vereinfacht komplexe Systeme, indem sie Details verbirgt
- Beispiele:
 - In der Programmierung von Maschinencode zu High-Level-Sprachen
 - In Netzwerken: durch **Schichtenmodelle** (Layer Models)

Netzwerk-Schichtenmodelle (Layer Models)

OSI-Modell (ISO/OSI)

- **7 Schichten** (standardisiert, konzeptionell)
- Unterteilt Aufgaben der Datenkommunikation in **klar definierte Ebenen**
- Ziel: **Interoperabilität** und **Modularität**

Internetmodell

- **5-Schichten-Modell** (praktischer, realitätsnäher)
- Nutzt im Kern die unteren 5 OSI-Schichten

Kommunikation im Netzwerk: Was ist ein Protokoll?

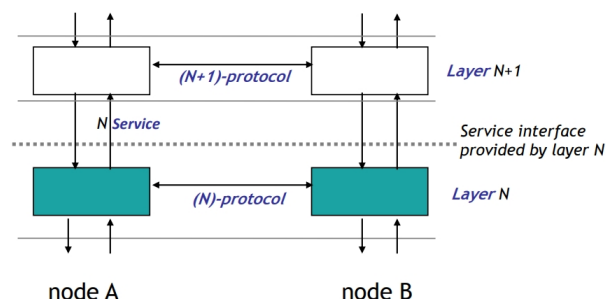
Ein **Protokoll** definiert **Format, Reihenfolge und Regeln** der Nachrichtenübertragung zwischen zwei oder mehreren Geräten

Funktionen eines Protokolls:

- Format & Struktur der Daten (Header, Payload, etc.)
- Verhalten beim Senden und Empfangen
- Beispiele für menschliche Protokolle: Begrüßung, Telefonanruf (Anruf → Antwort → Gespräch)

Schichtenarchitektur (Layered Architecture)

- Jede Schicht:
 - bietet **einen Dienst** (Service) für die darüberliegende Schicht
 - nutzt den Dienst der darunterliegenden Schicht
- Kommunikation erfolgt **zwischen gleichrangigen Entitäten (Peers)** derselben Schicht
- Vorteile:
 - Austauschbarkeit von Schichten
 - Klarheit & Wartbarkeit
- Nachteil: zusätzlicher Overhead (mehr Protokollinformationen)



Datenbegriffe in den Schichten (Data Units)

Begriff	Bedeutung
PDU	Protocol Data Unit - gesamte Einheit, die auf einer Schicht verarbeitet wird
SDU	Service Data Unit - „Nutzerdaten“, die von einer höheren Schicht übergeben wurden
PCI	Protocol Control Information - Steuerdaten für die Kommunikation auf derselben Ebene
Packet	Allgemeiner Begriff für Transporteinheit
Datagram	Verbindungslos gesendetes Packet
Frame	Datenpaket auf Sicherungsschicht mit finalem Header/Trailer
Cell	Feste kleine Pakete (z.B. bei ATM)

Verteilte Systeme & Netzwerke

Verteilte Maschine:

- Besteht aus **mehreren autonomen Systemen (AS)**, verbunden durch ein **Kommunikationssystem (CSS)**



Wichtige Eigenschaft:

- Das Kommunikationssystem **versteckt Komplexität** → Fehler, Heterogenität, Standorte, etc. werden **transparent**
- Verteilte Systeme setzen auf **funktionierenden Netzwerken auf**

1.7 OSI-Modell

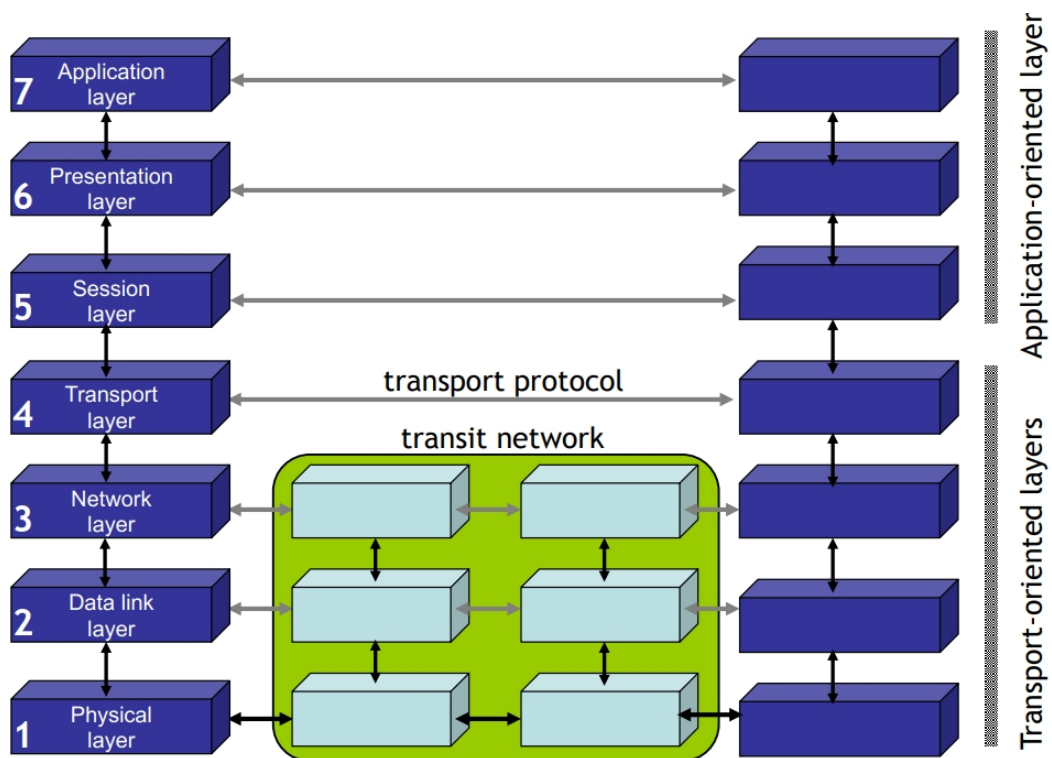
Was ist das OSI-Modell?

- Das **OSI-Modell** ist ein **theoretisches Modell**, das die Kommunikation zwischen zwei Computern in **sieben Schichten (Layers)** aufteilt
- Jede Schicht hat **eine bestimmte Aufgabe** und arbeitet mit der darunterliegenden zusammen - wie bei einer **Leiter**, von der Hardware bis zur Software

Warum OSI?

- **Vereinheitlichung:** Damit Geräte unterschiedlicher Hersteller zusammenarbeiten können
- **Abstraktion:** Komplexe Vorgänge werden in verständliche Teile zerlegt
- **Austauschbarkeit:** Einzelne Schichten können verändert werden, ohne das ganze System zu beeinflussen

Die 7 Schichten des OSI-Modells



1. Physical Layer (Physikalische Schicht)

„Wie werden einzelne Bits physisch übertragen?“

- Überträgt rohe Bits als elektrische Signale, Lichtimpulse oder Funkwellen
- Beispiel: Stecker, Kabel, Funkfrequenz, Spannung
- **Hardware-orientiert**
- **Protokoll-Beispiel:** RS232, Ethernet PHY

2. Data Link Layer (Sicherungsschicht)

„Wie machen wir aus den rohen Bits sinnvolle Datenrahmen und erkennen Fehler?“

- Erkennt & behebt Übertragungsfehler (z.B. durch Prüfsummen)
- Teilt den Datenstrom in **Frames**
- Flow-Control möglich (z.B. Sliding Window)
- Bei LANs:
 - **LLC (Logical Link Control)**: Fehlererkennung
 - **MAC (Media Access Control)**: Zugangskontrolle zum Übertragungsmedium (z.B. CSMA/CD)
- **Protokoll-Beispiel**: Ethernet, WLAN

3. Network Layer (Vermittlungsschicht)

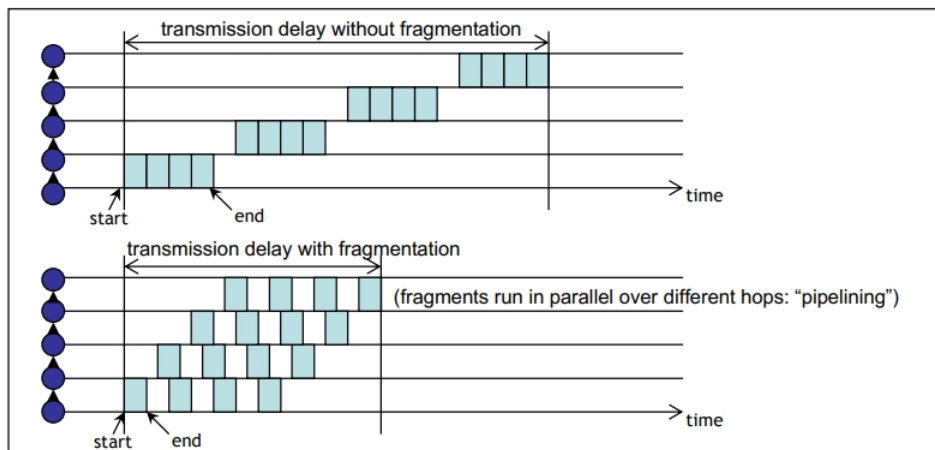
„Wie kommt ein Paket von A nach B - auch über mehrere Netze hinweg?“

- Routing: Pfadwahl durch das Netz
- Adressierung (z.B. IP-Adressen)
- Fragmentierung & Weiterleitung von Paketen
- **Protokoll-Beispiel**: IP, X.25

4. Transport Layer (Transportschicht)

„Wie garantieren wir eine sichere Ende-zu-Ende-Kommunikation zwischen Prozessen?“

- Verbindungsaufbau & -steuerung zwischen **Anwendungen/Prozessen**
- Fehlerkontrolle, Wiederholungen, Reihenfolge
- Multiplexing von Datenströmen (mehrere Verbindungen über einen Kanal)
- **Protokoll-Beispiel**: TCP (zuverlässig), UDP (schnell, aber unzuverlässig)



5. Session Layer (Sitzungssicht)

„Wie organisieren wir längere Kommunikationssitzungen?“

- Steuert Dialoge (z.B. wer darf wenn senden?)
- Checkpoints für Wiederherstellung nach Fehlern
- Heute selten genutzt, Funktionen oft in Transport-/Application-Schicht integriert
- **Beispiel**: frühere Dienste wie Teletext

6. Presentation Layer (Darstellungsschicht)

„Wie sorgen wir dafür, dass beide Seiten die Daten verstehen?“

- Übersetzt Datenformate (z.B. Zeichencodierung: ASCII ↔ Unicode)
- Syntaxvereinheitlichung
- **Datenkompression & Verschlüsselung** möglich
- **Beispiel:** ASN.1, BER, google Protobuf
- In modernen Netzwerken oft Teil der Anwendungsschicht

7. Application Layer (Anwendungsschicht)

„Wie interagieren Anwendungen mit dem Netzwerk?“

- Bietet Netzwerkdienste für Benutzerprogramme
- Enthält viele „Service Elements“ wie:
 - Dateiübertragung (FTP)
 - E-Mail (SMTP, X.400)
 - Web (HTTP)
- Praktisch: das, was du als Anwender siehst und nutzt

Wichtige Hinweise

- Viele Dienste werden **in mehreren Schichten** umgesetzt (z.B. Fehlerkorrektur)
- Das OSI-Modell ist **konzeptionell** - in der Praxis dominiert das Internet-Modell
- Dennoch: **OSI hilft enorm beim Verstehen komplexer Netzwerke**

Zwei Arten von Netzwerken / Protokollen

Es gibt **zwei grundlegende Arten**, wie Netzwerke Daten übertragen:

1. Connection-Oriented (CO)

- Verbindung wird vorher aufgebaut, wie bei einem Telefonat
→ Beispiel: **TCP**

2. Connectionless (CL)

- Daten werden einfach losgeschickt, ohne vorherige Verbindung
→ Beispiel: **UDP**

Conncection-Oriented (Verbindungsorientiert)

Ablauf in 3 Phasen:

1. **Verbindungsaufbau** (z.B. Handshake)
2. **Datenübertragung**
3. **Verbindungsabbau**

Beispiel: **Telefonanruf**

- Erst wählt du die Nummer
- Dann sprichst du
- Dann legst du auf

Wichtig:

- Auch wenn der Nutzer „verbunden“ ist, wissen die **Router im Internet** davon **nichts**
- Die Verbindung ist nur ein **Zustand an beiden Enden** (also auf deinem Gerät & dem Server)
- Das erlaubt bessere Dienste: **Zuverlässigkeit, Flusskontrolle, Staukontrolle**
- **Aber:** Das erzeugt **Overhead** (mehr Daten, mehr Speicher, mehr Komplexität)

Connectionless (Verbindungsfrei)

Ablauf:

1. **Nur Datenübertragung** - keine Vorbereitung, keine Aufräumphase

Beispiel: **Postkarte verschicken**

- Du schickst sie einfach ab, ohne zu fragen, ob jemand da ist
- Sie kommt vielleicht an - oder auch nicht

Eigenschaften:

- Kein Zustand nötig → **Skalierbar!**
- Aber: **Keine Garantie**, dass Daten ankommen oder geordnet sind
- Wird oft in Echtzeitanwendungen genutzt, wo Geschwindigkeit wichtig ist als Zuverlässigkeit (z.B. **Video-Streaming, Online-Gaming**)

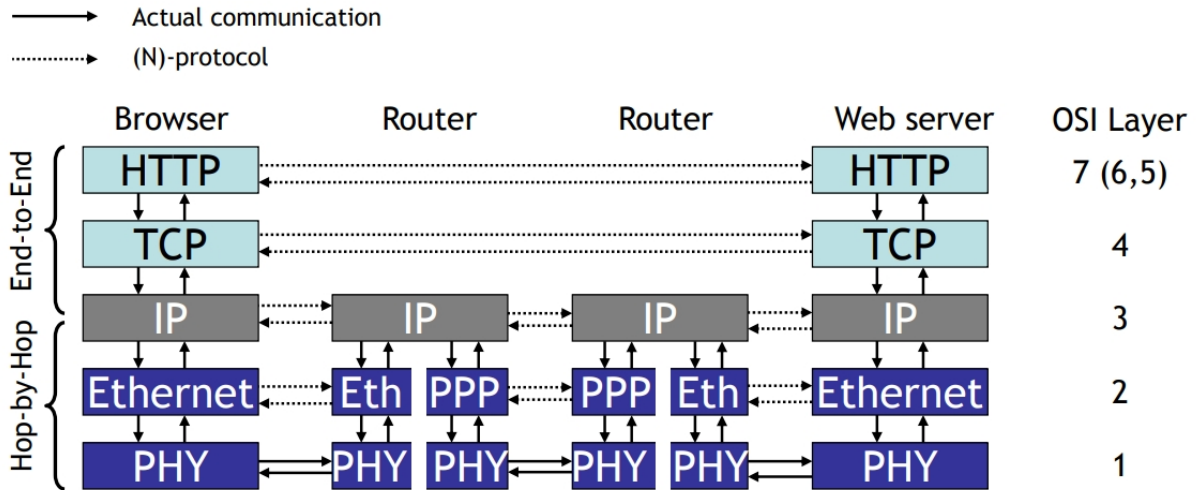
Was ist besser?

	Connection-Oriented (z.B. TCP)	Connectionless (z.B. UDP)
Vorteil	Zuverlässig, ordentlich	Schnell, einfach, skalierbar
Nachteil	Mehr Overhead	Keine Garantie
Beispiel	Web, E-Mail	Video, DNS, Voice Chat

Du kannst sogar **verbindungsorientierte Dienste auf verbindungslose Protokolle bauen** (z.B. QUIC auf UDP)

Beispiel: Was passiert, wenn du eine Webseite aufrufst

Du öffnest den Browser → gibst eine URL ein → und dann passiert viel im Hintergrund



OSI-Schicht	Aufgabe
7 (6,5) Anwendung	HTTP-Anfrage erstellen, Antwort anzeigen
4 - Transport	TCP-Verbindung aufbauen, Daten in Reihenfolge bringen
3 - Netzwerk	IP: Routing durch Internet (Router zu Router)
2 - Sicherung	Ethernet/WLAN: Rahmen für jedes Paket
1 - Bitübertragung	Sendet die Bits über Kabel oder Funk

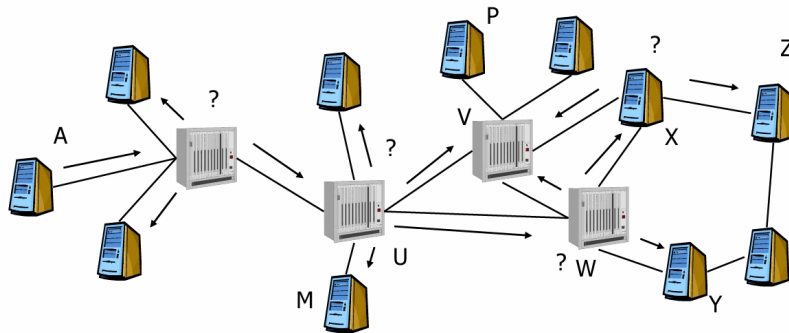
2 Routing

2.1 Introduction

Wohin werden Pakete geschickt?

Wenn ein Paket bei einem Router (auch „Switching Element“) ankommt, muss dieser entscheiden: „An welchen Nachbarn leite ich dieses Paket weiter, damit es sein Ziel erreicht?“

Diese Entscheidung basiert auf Routing-Informationen, nicht auf Zufall



Grundbegriffe

Weiterleitung (Forwarding)

- Der Vorgang, bei dem ein Router ankommendes Paket an den passenden Ausgang (nächsten Knoten) weiterleitet
- Vergleich: „An welcher Ausfahrt verlasse ich den Kreisverkehr?“

Routing

- Der Prozess, mit dem berechnet wird, wie ein Paket durch das Netzwerk gelangen soll
- Vergleich: „Wie plane ich die beste Route von A nach B?“

Routing erstellt die Karte,
Forwarding nutzt sie für jeden einzelnen Schritt

Wie weiß der Router, wohin?

Router benutzen eine sogenannte Routing-Tabelle, die durch Routing-Algorithmen aufgebaut wird:

1. Router kommunizieren untereinander über Routing-Protokolle (z.B. OSPF, BGP, RIP)
2. Diese Protokolle tauschen Informationen über die Netzwerkstruktur aus
3. Jeder Router baut daraus seine eigene Routing-Tabelle - wie ein Navi
4. Wenn ein Paket ankommt:
 - Der Router schaut auf die Zieladresse
 - Er sucht in der Tabelle den besten nächsten Knoten (next hop)
 - Dann leitet er das Paket dorthin weiter

Große Netzwerke bauen

Einfach Verfahren wie:

- Flooding (Paket an alle Nachbarn schicken),
- Hot-Potato (schnellstmögliche Weiterleitung ohne Plan),

...sind ineffizient in großen Netzwerken

Sie erzeugen zu viel Datenverkehr oder führen zu schlechten Routen

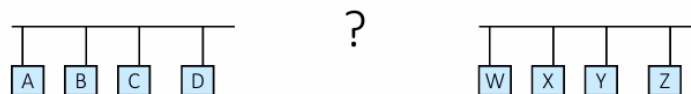
Deshalb braucht man strukturierte Routing-Algorithmen, um „beste Wege“ zu berechnen

Analogie: Reisen

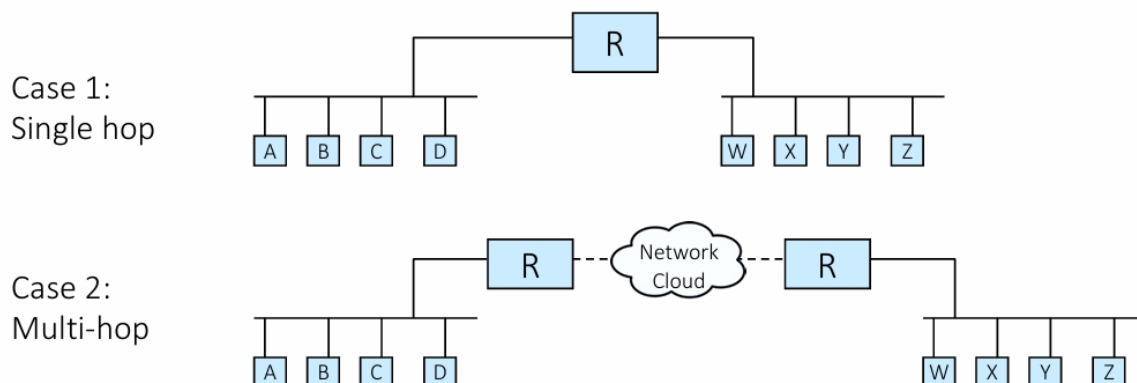
Begriff	Netzwerk	Analogie
Routing	Routenplanung	Reisen planen (z.B. mit Google Maps)
Forwarding	Weiterleiten	Nächste Abzweigung nehmen
Routing-Tabelle	Wegweiser/Karte	Navigationskarte

Forwarding

Question: How do you get a packet from one network to another?



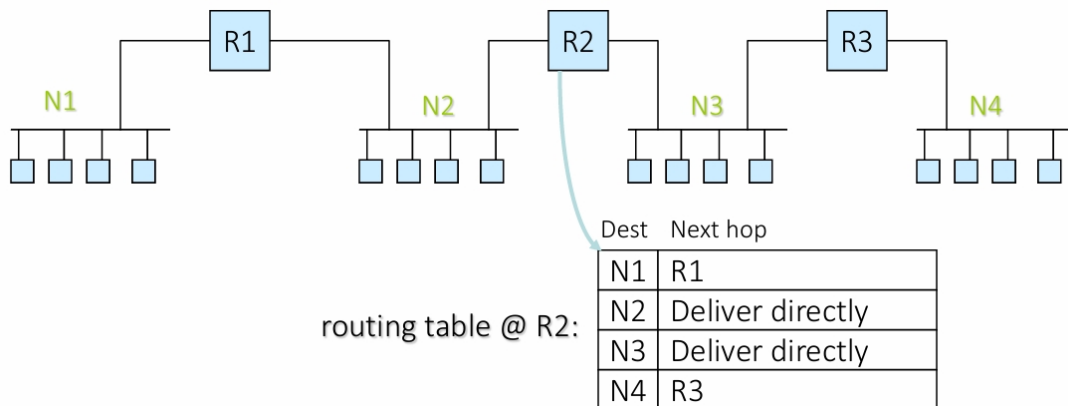
Answer: with a **router**¹ (or a series of them)



Begriffe

- **Router:** Vermittelt Pakete zwischen Netzwerken
- **Bridge / Switch / Hub / Gateway:** Ähnliche Geräte auf anderen Ebenen oder mit anderen Aufgaben:
 - Bridge: verbindet zwei Netzwerksegmente auf Layer 2 (Data Link Layer)
 - Switch: arbeitet wie ein Bridge, aber effizienter (Multiport-Bridge)
 - Hub: einfache „Verteilerdose“, keine Intelligenz
 - Gateway: Verbindung zu anderen Netzwerktypen/Protokollen (oft auf höheren Ebenen)

Beispiel: Routing-Tabelle in Aktion



Was heißt das konkret?

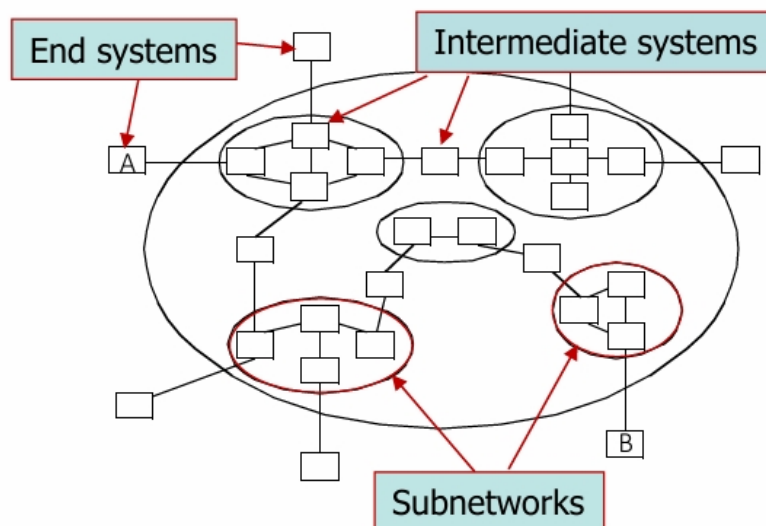
- „Direkt zustellbar“ bedeutet: Das Zielnetzwerk (N2 oder N3) ist direkt an R2 angeschlossen. R2 kennt die genauen Schnittstellen und kann das Paket ohne Umweg direkt in das Netzwerk schicken
- „R1“ bzw. „R3“ als „Nächster Hop“ bedeutet: R2 weiß, dass er das Paket an einen benachbarten Router weiterleiten muss, der näher am Ziel ist. Dieser andere Router übernimmt dann die weitere Weiterleitung

Hinweis: Die echte Routing-Tabelle enthält IP-Adressen, Netzmasken, Flags usw., nicht nur solche symbolischen Namen

Routing

Was ist Routing - und wie funktioniert es?

Routing ist eine Kernaufgabe in Computernetzwerken, insbesondere im sogenannten Netzwerk-Layer (Schicht 3 im OSI-Modell). Dabei geht es darum, einen geeigneten Pfad (Route) für Datenpakete von einer Quelle (z.B. Computer A) zu einem Ziel (z.B. Computer B) zu finden - über eventuell viele Zwischenstationen (Router)



Man unterscheidet zwei Aufgaben bei Routern:

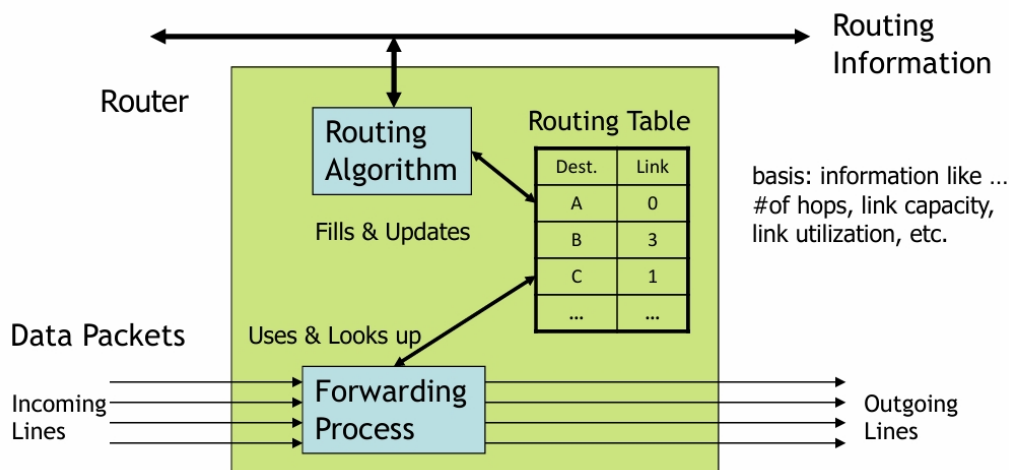
1. Routing (Wegwahl):

- Ein Router entscheidet grundsätzlich, welcher Weg (Pfad) zu einem Ziel sinnvoll ist
- Dazu verwendet er **Routing-Algorithmen**
- Diese Algorithmen berücksichtigen oft Metriken wie:
 - Anzahl der Hops (also wie viele Router der Pfad durchläuft)
 - Auslastung oder Kapazität einer Verbindung
 - Verzögerung (Delay)

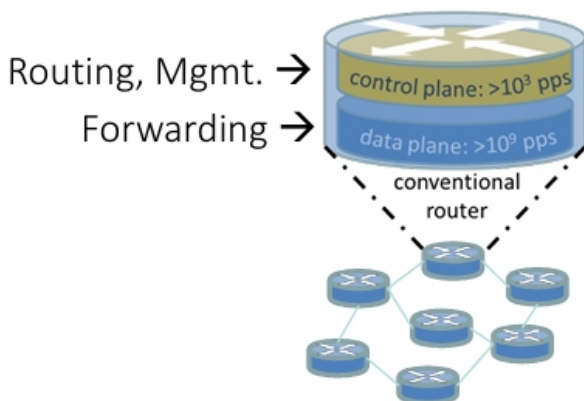
Das Ziel ist meist: Ein möglichst „günstiger“ oder „schneller“ Pfad (oft geringste Kosten)

2. Forwarding (Weiterleitung)

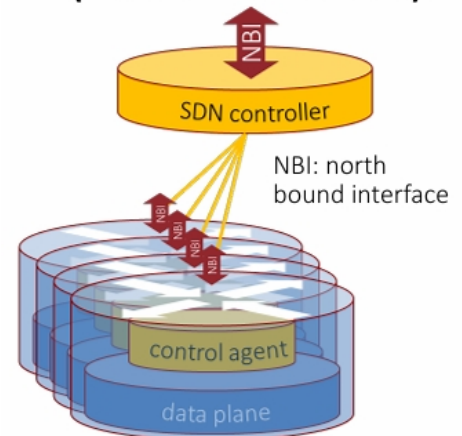
- Das ist die konkrete Aktion, wenn ein Paket eintrifft:
 - Der Router schaut in seiner **Routing-Tabelle** nach,
 - findet den passenden Ausgangsport („nächster Hop“)
 - und leitet das Paket dorthin weiter



Router = “two kinds of computers”



The idea of SDN (SW-defined networks)



Wie entsteht eine Routing-Tabelle?

Diese Tabelle wird von einem **Routing-Algorithmus** aufgebaut und regelmäßig aktualisiert - entweder:

- periodisch (z.B. alle 30 Sekunden), oder
- bei einer Änderung im Netzwerk (z.B. wenn ein Router ausfällt)

Unterschied: Verbindungslos vs. verbindungsorientiert

- In verbindungslosen Netzwerken (wie dem heutigen Internet) wird Routing unabhängig für jedes einzelne Paket gemacht
 - Jedes Paket enthält die Zieladresse
 - Der Router entscheidet jedes Mal anhand der Adresse, wohin es geht
- In verbindungsorientierten Netzwerken (z.B. ATM, MPLS) wird beim Verbindungsaufbau der Pfad einmal bestimmt
 - Danach bekommen die Pakete eine ID (Circuit ID)
 - Der Router muss sich nur merken: „Paket mit ID X → weiter über Port Y“

Virtual Circuit (CONS) vs Datagram (CLNS):

- **Virtual Circuit (CONS):**
 - In diesem Modell wird eine **Verbindung** zwischen den Endsystemen vor der Datenübertragung aufgebaut
 - **Routing-Algorithmus** wird **während der Verbindungsaufbauphase** durchgeführt
 - Während der Kommunikation wird nicht die Zieladresse verwendet, sondern eine **Verbindungs-ID** oder **Circuit ID**. Diese ID ist eine Art temporäre Adresse, die die gesamte Sitzung identifiziert
 - Alle Zwischenstationen (Router, Switches) wissen über die Verbindung, da sie während des Verbindungsaufbaus festgelegt wird
- **Datagram (CLNS):**
 - Hier handelt es sich um ein **verbindungsfreies Modell**
 - Jede Nachricht (Paket) wird **unabhängig** voneinander versendet
 - **Routing** wird normalerweise **nicht bei jeder Ankunft eines Pakets** durchgeführt, da dies eine enorme Rechenlast verursachen würde
 - Stattdessen wird das Routing üblicherweise **periodisch oder nach einer Netzwerkanpassung** durchgeführt. Das heißt, wenn sich das Netzwerk ändert oder wenn ein Fehler auftritt, wird das Routing neu berechnet und die Routing-Tabellen aktualisiert
 - Das Internet verwendet diesen **datagrammorientierten Ansatz** (z.B. durch **IP-Routing**)

Unterschiede zwischen network service und transport service (NS vs. TS):

- **COTS (Connection-Oriented Transport Service):**
 - Nur die **Endsysteme** wissen, dass sie miteinander verbunden sind
 - Ein Beispiel dafür ist das **TCP** (Transmission Control Protocol). Es stellt sicher, dass die Daten zuverlässig über die Verbindung übertragen werden und dass die Endsysteme über den Status der Verbindung Bescheid wissen
- **CONS (Connection-Oriented Network Service):**
 - Alle **Zwischenstationen** (Router und Switches) wissen über die Verbindung Bescheid und verwalten die **Verbindungs-ID**
 - Ein Beispiel ist das **ATM (Asynchronous Transfer Mode)** oder **Frame Relay**, bei denen eine virtuelle Verbindung zwischen den Endpunkten eingerichtet wird und alle Zwischenstationen (z.B. Router) diese Verbindung kennen

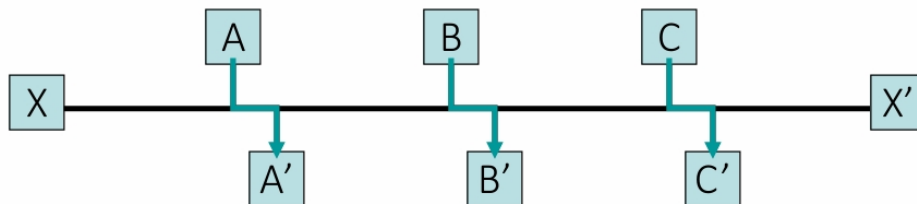
Zielkonflikte im Routing

Routingalgorithmen sollen möglichst viele gute Eigenschaften erfüllen:

Eigenschaft	Bedeutung
Korrektheit	Liefert immer gültige Wege
Einfachheit	Gut implementierbar, wenig Rechenaufwand
Robustheit	Funktioniert auch bei Ausfällen
Stabilität	Keine dauernden Schwankungen
Fairness	Kein Teilnehmer wird benachteiligt
Optimalität	Nutzung der Netzressourcen ist effizient

Diese Eigenschaften stehen aber oft im Widerspruch. Beispiel:

- Es wäre optimal, wenn nur $A \rightarrow A'$, $B \rightarrow B'$ und $C \rightarrow C'$ die gesamte Bandbreite einer Leitung nutzen



- Aber das wäre unfair gegenüber anderen Verbindungen (z.B. $X \rightarrow X'$), die dann keinen Durchsatz bekommen

Häufig verwendete Metrik: Minimale Hops

In der Praxis versuchen viele Routingprotokolle einfach die Anzahl der Hops zu minimieren:

- Das führt oft zu akzeptablen Ergebnissen
- Aber es garantiert nicht immer den optimalen Weg (z.B. im Sinne von Latenz oder Auslastung)

Klassifizierung der Routing-Algorithmen

Routing-Algorithmen lassen sich grob in zwei Hauptkategorien einteilen:

- **Nicht-adaptative Routing-Algorithmen:** Diese Algorithmen treffen ihre Entscheidungen ohne Berücksichtigung des aktuellen Netzwerkzustands. Sie basieren auf vordefinierten Routen oder einfach Strategien wie **Fluten** oder **Pro-Konfiguration**
- **Adaptive Routing-Algorithmen:** Diese Algorithmen berücksichtigen den aktuellen Zustand des Netzwerks bei ihren Entscheidungen. Sie reagieren dynamisch auf Änderungen, wie z.B. Netzwerküberlastung oder Ausfälle. Beispiele sind **Distance-Vector Routing** oder **Link-State Routing**

Missverständnis über adaptive Routing-Algorithmen

- Viele glauben, dass „adaptive“ Routing-Algorithmen die Netzwerklast kontinuierlich anpassen. Dies ist jedoch schwierig und kann sogar zu mehr Problemen führen, weil sich der Netzwerkverkehr schnell ändern kann. Zu viel Traffic zur Anpassung kann das Netzwerk weiter belasten. Ein gutes Beispiel: Wenn eine Leitung überlastet ist, kann sie so behandelt werden, als wäre sie ausgefallen, um den Traffic um die problematische Strecke herumzuleiten

2.2 Nicht-adaptive Routing Algorithmen

Flooding (Fluten)

Flooding ist eine einfach, nicht-adaptive Strategie:

- **Funktionsweise:** Jedes eingehende Paket wird an jeden benachbarten Router weitergeleitet, außer dem Router, vor dem es kam, und es wird nicht an das Endziel weitergeleitet, wenn es dieses bereits erreicht hat
- **Problem:** Flooding erzeugt viele Duplikate von Pakete, die das Netzwerk stark belasten können

Lösungen zur Reduzierung von Duplikaten:

1. **TTL (Time To Live):** Jedes Paket hat eine TTL (Lebensdauer), die von den Routern auf jeder Weiterleitung dekrementiert wird. Wenn die TTL 0 erreicht, wird das Paket verworfen. Die TTL wird idealerweise basierend auf der Entfernung vom Ursprung zum Ziel initialisiert
2. **Sequenznummern:** Ein Router fügt eine Sequenznummer hinzu, die jedes Paket eindeutig kennzeichnet. Routern wird es dann ermöglicht, Pakete zu erkennen, die sie bereits weitergeleitet haben, und sie zu verwerfen

Flooding: Mögliche Anwendungen

Flooding kann in einigen Szenarien sinnvoll sein, z.B. in dynamischen Netzwerken, wo die Anpassung an Netzwerkverkehr zu langsam oder zu ressourcenintensiv wäre

Beispiele für Anwendungen:

- **Krisensituationen:** Bei Naturkatastrophen oder Ausfällen, wo schnelle Netzwerkverbindungen nötig sind. Hierbei könnte Flooding sicherstellen, dass auch bei Ausfällen von Routern die Daten ihre Zielsysteme erreichen
- **Verteilte Datenbanken:** Bei der gleichzeitigen Aktualisierung mehrere Datenbanken könnte Flooding verwendet werden, um eine einzige Nachricht an alle relevanten Systeme zu senden
- **Netzwerke mit häufigen Topologieänderungen:** In mobilen Netzwerken (wie Ad-hoc-Netzen oder Peer-to-Peer-Netzen) wird Flooding genutzt, um die Kommunikation zu ermöglichen, auch wenn sich die Netzwerkstruktur ständig verändert

Statisches Routing

Statisches Routing ist eine nicht-adaptive Strategie, bei der alle Router mit festen Routen zu allen möglichen Zielen vorab konfiguriert sind

- **Vorteil:** In einem stabilen, vorhersehbaren Netzwerk funktioniert statisches Routing sehr gut, weil es keine komplexen Berechnungen erfordert
- **Nachteile:** Bei häufigen Änderungen im Netzwerk (z.B. Routerausfällen, Netzwerkkapazitäten oder Verkehrsschwankungen) ist statisches Routing problematisch. Hier können adaptive Algorithmen besser reagieren und die Routen dynamisch anpassen, um das Netzwerk effizient zu nutzen

Problem mit nicht-adaptiven Algorithmen

- Wenn das Netzwerk stark verändert wird oder die Verkehrsmuster schwanken, würde ein nicht-adaptives System enorme Ressourcen wie große Kabel und Router erfordern, um das Netzwerk am Laufen zu halten - was ineffizient ist
- Netzwerklast ist oft „burstig“ - d.h. sie variiert stark im Laufe der Zeit. Im Gegensatz zu physischen Flüssen wie Wasser oder Elektrizität, die automatisch Lasten auf alternativen Pfaden ausgleichen, muss ein Netzwerk aktiv steuern, wie der Traffic verteilt wird

2.3 Adaptive Routing Algorithms

Adaptive Routing bedeutet, dass Routing-Entscheidungen auf Basis des aktuellen Netzwerkzustands getroffen werden. Es gibt drei Haupttypen:

Centralized Adaptive Routing

Zentrale Steuerung durch ein Routing Control Center (RCC)

- **Funktionsweise:**
 - Alle Router senden regelmäßig Informationen über ihre Links (z.B. Auslastung, Verzögerung, Verfügbarkeit) an das RCC
 - Das RCC konstruiert damit ein vollständiges Netzwerkgraph-Modell
 - Es berechnet optimale Routen (z.B. mittels Dijkstra-Algorithmus) und sendet die Routen an die einzelnen Router zurück
- **Vorteile:**
 - Globale Sicht auf das Netzwerk ermöglicht gute Routenentscheidungen
 - Einheitliche zentrale Verwaltung vereinfacht bestimmte Aufgaben
- **Nachteile:**
 - **Single Point of Failure:** Fällt das RCC aus, wird das Routing nicht mehr aktualisiert
 - **Skalierungsprobleme:** Bei großen Netzwerken überfordert die zentrale Instanz
 - **Verzögerungen:** Nahe Router melden und erhalten Änderungen schneller als entfernte
 - **Bottleneck-Risiko:** Viel Verkehr konzentriert sich um das RCC

Isolated (Local) Adaptive Routing

Jeder Router trifft Entscheidungen nur auf Basis seiner eigenen Informationen

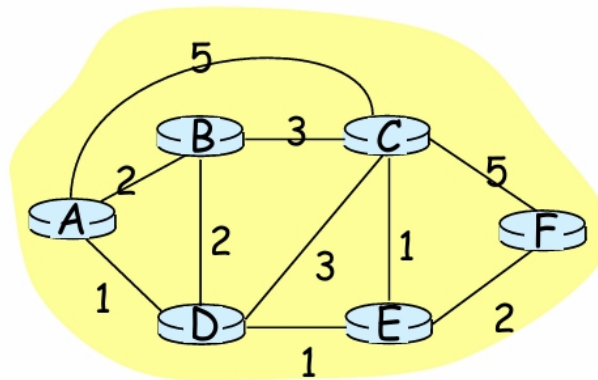
- **Eigenschaften:**
 - Keine Kommunikation mit anderen Routern nötig
 - Routenwahl basiert auf lokalen Kriterien (z.B. Queue-Länge an den Ports)
- **Beispiele:**
 - **Hot Potato Routing:**
 - * Ziel: Pakete so schnell wie möglich loswerden, egal in welche Richtung
 - * Oft wird der Ausgang mit der kürzesten Warteschlange gewählt
 - * Vorteil: sehr einfach, geringe Verzögerung
 - * Nachteil: Pakete können ineffizient durch das Netzwerk irren
 - **Backward Learning:**
 - * Router lernen aus beobachtetem Verkehr
 - * Jeder Router merkt sich für jede Quelladresse, von welchem Interface das Paket kam und wie hoch der Hop Count war
 - * Wenn ein Paket mit niedrigerem Hop Count ankommt → bessere Route entdeckt

- **Vorteil:**
 - Kein Overhead durch Kommunikation mit anderen Routern
- **Nachteil:**
 - Potenziell suboptimale oder instabile Routen

Distributed Adaptive Routing

Router tauschen regelmäßig Informationen untereinander aus

- **Ziel:**
 - Finden eines „guten“ Pfads (z.B. mit minimalem Hop Count oder minimaler Verzögerung) vom Sender zum Empfänger
- **Graph-Modell:**
 - Router = Knoten im Graphen
 - Links (Hops) = Kanten mit bestimmten Kosten (z.B. Verzögerung, Auslastung, Bandbreite)



- **Klassifizierung:**
 - **Distance Vector Routing** (z.B. RIP, BGP):
 - * Jeder Router kennt nur seine Nachbarn
 - * Router tauschen regelmäßig Vektoren mit Distanzen zu Zielen aus
 - * Iterativer Prozess: Jeder Router aktualisiert seine Tabelle basierend auf den Nachbarn
 - **Link State Routing** (z.B. OSPF):
 - * Jeder Router sammelt vollständige Topologie-Informationen
 - * Broadcastet diese Informationen an alle Router
 - * Jeder Router berechnet lokal den optimalen Pfad (z.B. via Dijkstra)
 - **Statisch vs. Dynamisch:**
 - * **Statisch:** Routen ändern sich selten
 - * **Dynamisch:** Routen passen sich regelmäßig an (z.B. bei Änderung der Linkkosten)
 - **Vorteile:**
 - * Flexibel, reagiert auf Netzwerkänderungen

- * Kann Last balancieren, Überlastungen vermeiden

– **Nachteile:**

- * Mehr Overhead durch Informationsaustausch
- * Potenziell instabile Routen bei schneller Änderung der Bedingungen (z.B. Routing Loops, Count-to-Infinity Problem bei Distance Vector)

2.4 Distance Vector Routing (DVR)

Grundprinzip

- Jeder Router kennt nur die **Kosten zu seinen direkten Nachbarn**
- Jeder Router erstellt und pflegt eine **Distance Table**, in der für jede Destination und jeden Nachbarn die Gesamtkosten vermerkt sind
- **Distance Vector** = für jeden Zielknoten: aktuell geringste bekannte Kosten
- Durch den Austausch dieser Distance Vectors mit Nachbarn werden die Routen iterativ verbessert

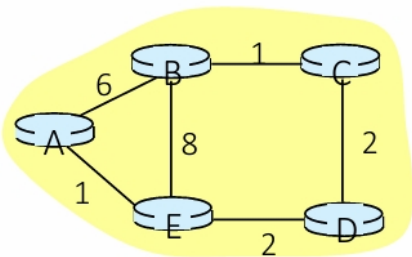
Distance Table

- Eintrag $D_X(Y, Z)$: Kosten von Router X zu Ziel Y über Nachbarn Z:

$$D^X(Y, Z) = c(X, Z) + \min_w \{D^Z(Y, W)\}$$

- Ziel: Für jedes Ziel Y die Route mit minimalen Kosten (über irgendeinen Nachbarn Z) zu finden

Example for Distance Table:



$$\begin{aligned} D^E(C, D) &= c(E, D) + \min_w \{D^D(C, w)\} \\ &= 2 + 2 = 4 \end{aligned}$$

$$\begin{aligned} D^E(A, D) &= c(E, D) + \min_w \{D^D(A, w)\} \\ &= 2 + 3 = 5 \quad \text{loop!} \end{aligned}$$

$$\begin{aligned} D^E(A, B) &= c(E, B) + \min_w \{D^B(A, w)\} \\ &= 8 + 6 = 14 \end{aligned}$$

		cost to destination via		
destination	$D^E()$	A	B	D
	A	1	14	5
	B	7	8	5
	C	6	9	4
	D	4	11	2

Derive Routing Table from Distance Table

		Cost to destination via		
Destination	D ^E ()	A	B	D
	A	1	14	5
	B	7	8	5
	C	6	9	4
	D	4	11	2

Destination	Outgoing link to use, cost	
	A	A,1
	B	D,5
	C	D,4
	D	D,2

E:	
A	1
B	5
C	4
D	2

distance table (matrix)
for route finding (computing)

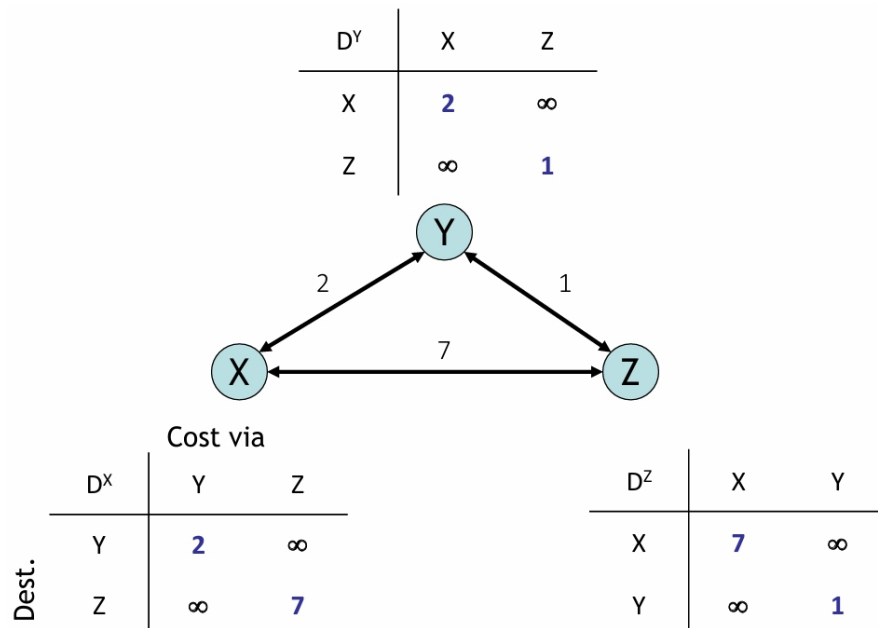


routing table
for packet forwarding

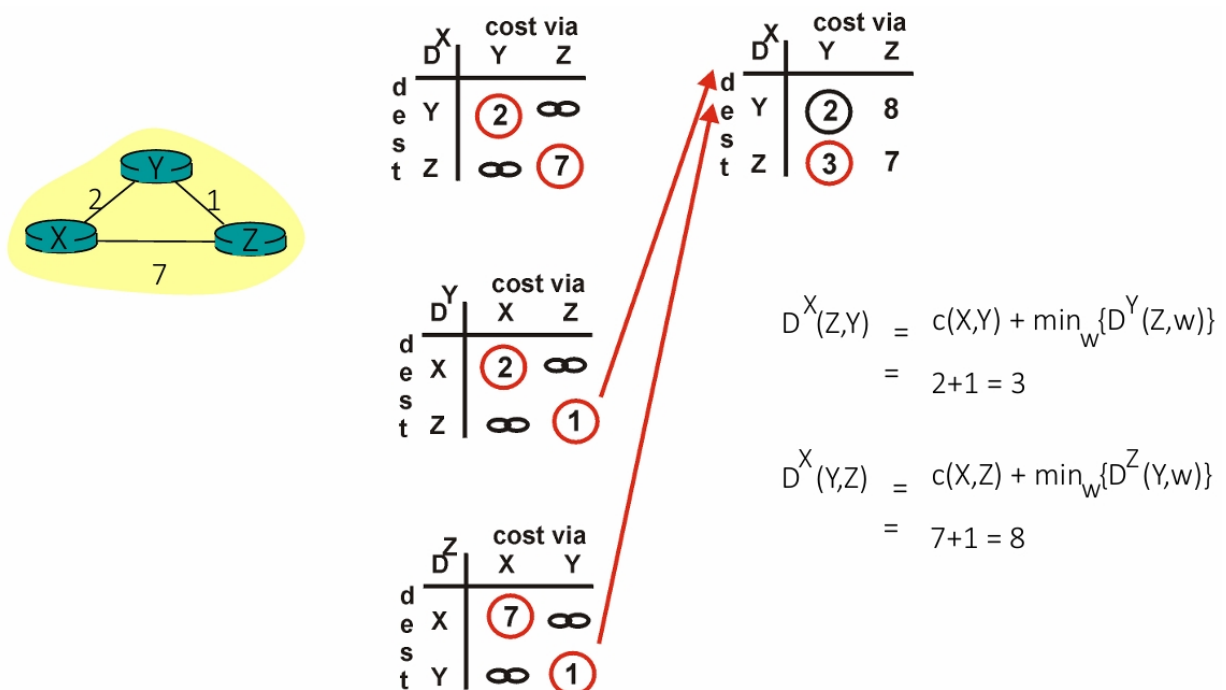


distance vector
for neighbor notification

Example: Initialization of Distance Tables

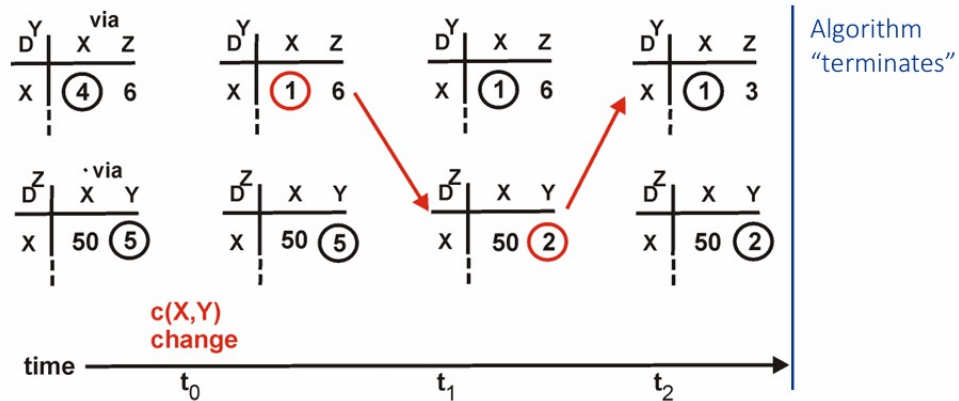
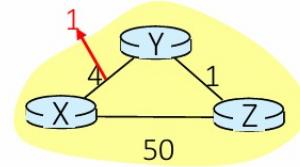


Example: Distance Vector Algorithm

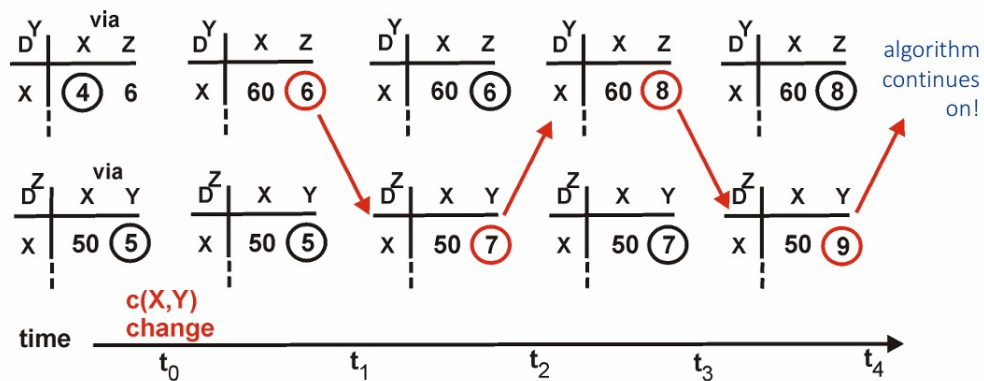
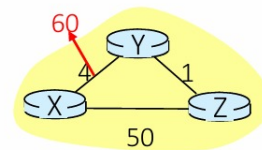


DVR: Reaction to Link Cost Changes

- **Gute Nachrichten (niedrige Kosten)** verbreiten sich schnell: neue Pfade werden schnell übernommen



- **Schlechte Nachrichten (höhere Kosten)** → „Count-to-Infinity“-Problem:
 - Schlechte Routen werden langsamer verbreitet
 - Kann zu **Routing Loops** führen
 - Lösungsmöglichkeiten: Split Horizon, Poison Reverse, Triggered Updates



Was ist das Count-to-Infinity-Problem?

Beim Distance Vector Routing kann es passieren, dass sich zwei oder mehr Knoten gegenseitig falsche Informationen schicken - z.B. wenn eine Verbindung ausfällt. Dadurch „glauben“ sie, es gäbe noch einen Pfad, obwohl es keinen mehr gibt. Ihre Schätzungen für die Distanz steigen dann schrittweise → theoretisch bis ∞

DVR - Poisoned Reverse

Technik zur Bekämpfung von Count-to-Infinity:

Wenn ein Knoten Z über seinen Nachbarn Y zu einem Ziel X routet, dann teilt er Y absichtlich mit:

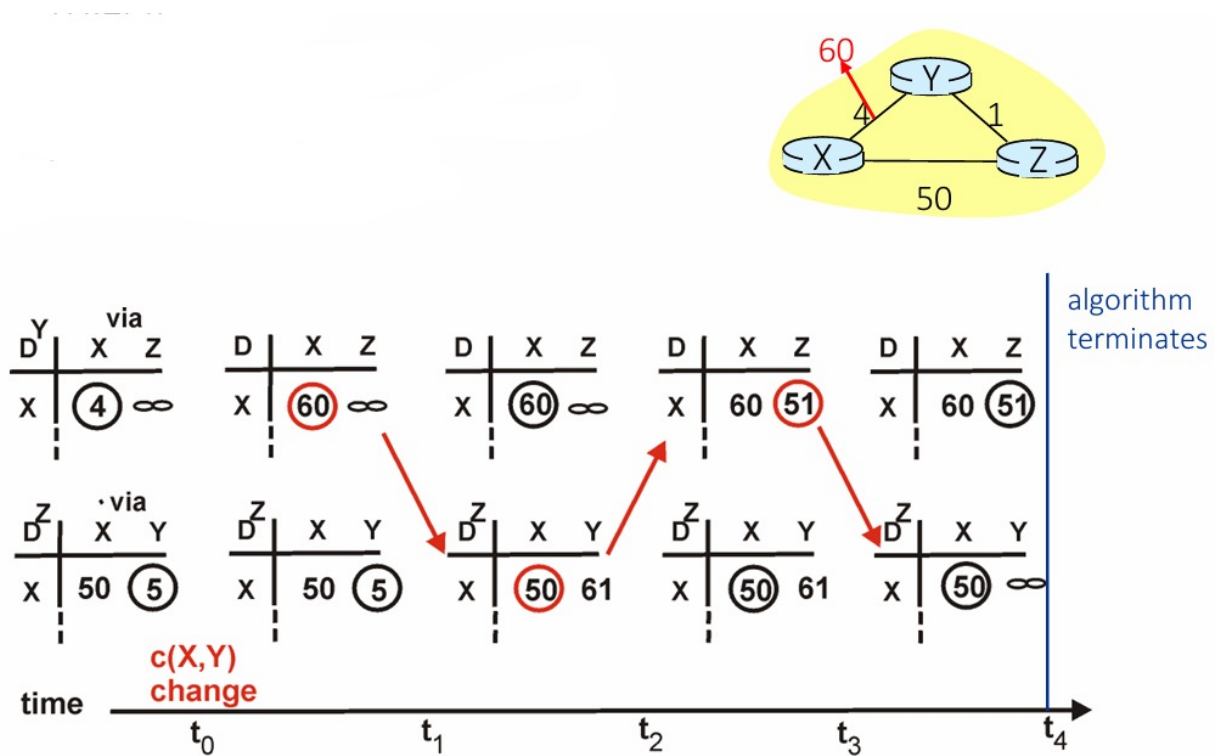
→ „Meine Distanz zu X ist ∞ “

So verhindert Z, dass Y wiederum zu X über Z routet (was zu einer Schleife führen würde)

Beispiel:

- X – Y – Z
- Angenommen, der Pfad Z → X ist über Y
- Dann teilt Z Y mit: $D^Z(X) = \infty$

→ Dadurch weiß Y: „ich sollte nicht zu X über Z gehen“



Warum funktioniert Poisoned Reverse nicht immer?

In kleinen Netzen funktioniert es meist. Aber in größeren Netzen können mehrere Router sich gegenseitig „vergiften“, ohne dass ein echter Alternativpfad existiert

Poisoned Reverse verhindert Routing-Schleifen, aber nur zwischen zwei direkt verbundenen Routern. Es funktioniert nicht mehr zuverlässig, wenn das Netz mehr als zwei Hops umfasst und mehrere Router beteiligt sind, die sich gegenseitig beeinflussen können - ohne direkte Verbindung zueinander Beispiel:

Netzwerk: A – B – C – D

- Jeder Link hat Kosten 1
- Alle Router kennen anfangs die besten Wege zu allen Zielen

Schritt-für-Schritt - der Fehlerfall:

1. **Link C-D fällt aus** (z.B. C erkennt: D nicht erreichbar)
2. C entfernt D aus seiner Routingtabelle
3. C sagt **B**, dass D nicht mehr erreichbar ist
4. **ABER:** B hat noch **veraltete Information** - B glaubt, D sei über C erreichbar (Kosten 2)
 - B hat vielleicht noch nicht das neue Update von C bekommen
 - Oder: B erhält das Update von C, aber **sieht gleichzeitig**, dass **A** noch sagt: „Ich erreiche D über B mit Kosten 3“
 - B denkt: „Wenn A einen Weg zu D kennt (über mich), dann kenne ich auch einen Weg - über A“
→ Das ist der Trugschluss
5. B meldet **A**, dass D über B erreichbar ist
6. A sagt: „Ah, D über B - gut!“ und merkt sich D über B mit Kosten 3
7. Jetzt sagt A zu B: „Ich kenne einen Weg zu D über dich - kostet 3“
8. B denkt nun: „A kennt D - dann kenne ich D über A - mit Kosten 4“
9. usw.

DVR - Split Horizon

Verwandte Technik:

- Ein Knoten teilt einem Nachbarn **gar nicht** mit, welche Routen er **über genau diesen Nachbarn** gelernt hat

Beispiel:

- X lernt Route zu Z über Y
- Dann sagt X dem Y **nichts** über Route zu Z

→ Ziel: Y soll X nicht als Rückweg zu Z in Betracht ziehen

2.5 Link-State Routing (LSR)

Link-State Routing (LSR) ist ein Routing-Algorithmus, bei dem jeder Knoten im Netzwerk die vollständige Topologie kennt - im Gegensatz zu Distance Vector Routing (DVR), bei dem Knoten nur Infos über ihre Nachbarn haben

Grundidee von Link State Routing

Jeder Knoten führt folgende Schritte aus:

1. **Link-State Erkennung:**
Er misst die Kosten zu seinen direkten Nachbarn (z.B. Latenz, Bandbreite, Anzahl Hops)
2. **Link-State-Verteilung (Flooding):**
Er verschickt sogenannte **Link-State-Pakete (LSPs)** mit Infos über alle seine direkten Verbindungen an alle anderen Router im Netz
3. **Topologie-Rekonstruktion:**
Jeder Router speichert alle empfangenen LSPs → damit kann er das gesamte Netzwerk als Graph rekonstruieren
4. **Kürzeste-Wege-Berechnung (Dijkstra):**
Mit Dijkstra's Algorithmus berechnet der Router den kürzesten Pfad zu jedem anderen Knoten
5. **Routing-Tabelle-aktualisieren:**
Er erzeugt daraus die Tabelle, die bestimmt, welchen Ausgangsport (next hop) er für jedes Ziel verwenden soll

Vergleich DVR vs. LSR

Eigenschaft	Distance Vector (DVR)	Link-State Routing (LSR) - Dijkstra
Prinzip	Iterative, verteilt	Global, zentralisiertes Wissen
Infoverteilung	Nur zwischen Nachbarn	Link-State-Pakete werden geflutet
Routingfehler	Count-to-Infinity möglich	Weniger Fehleranfällig
Schleifen	Möglich	Sehr unwahrscheinlich
Algorithmen	Bellman-Ford	Dijkstra
Konvergenz	Langsam, kann oszillieren	Schnell, $O(n^2)$
Robustheit	Fehler in Nachbarn beeinflussen viele Knoten	Fehler beschränkt auf falsche Link-State-Infos
Nachrichtenkomplexität	Geringer (aber bei Fehlern evtl. viele Iterationen)	Höher (wegen Flooding), aber deterministisch

2.6 Hierarchical Routing

Motivation für Hierarchical Routing

- **Bisherige Annahmen (idealisiert):**
 - Alle Router sind gleich
 - Das Netzwerk ist „flach“: alle Router kennen sich
- **In der Praxis jedoch:**
 - Das Internet ist riesig: Milliarden von Zielen
 - Jeder Router kann nicht alle Ziele kennen
 - Routing-Updates zwischen allen Routern wäre zu viel Traffic
 - Verschiedene Betreiber (Provider, Firmen) wollen Kontrolle über ihr Routing behalten

Autonomous Systems (AS)

Ein Autonomous System (AS) ist:

- Eine Sammlung von Routern unter gemeinsamer administrativer Kontrolle
- Im Internet bestehen ca. 120000 AS (Stand 2025)
- Es gibt drei Typen:
 - **Stub AS:** Nur ein Verbindungsweg ins Internet (z.B. kleine Firma, Uni mit einem einzigen ISP)
 - * Verbindet sich nur mit einem anderen AS
 - * Verwendet BGP nur für eingehenden und ausgehenden Verkehr
 - * Leitet keinen Transitverkehr für andere AS weiter
 - * Einfachste Art von AS
 - **Multihomed AS:** Mehrere Internetverbindungen, aber leitet keinen Transitverkehr weiter (z.B. Firma mit zwei ISPs für Redundanz)
 - * Hat Verbindungen zu mehreren AS, z.B. zu mehreren ISPs
 - * Verwendet BGP, um Verfügbarkeit zu erhöhen (Redundanz, Ausfallsicherheit)
 - * Leitet jedoch keinen Verkehr weiter, der nicht für es selbst bestimmt ist
 - * Nimmt am BGP teil, filtert aber Transitverkehr
 - **Transit AS:** Internetanbieter, leitet Datenverkehr für andere AS weiter (Internet Service Provider z.B. Telekom)
 - * Verbindet sich mit mehreren AS
 - * Leitet Verkehr weiter, auch wenn der Verkehr nicht für das eigene Netz bestimmt ist
 - * Bietet Transitsdienste an → andere AS bezahlen dafür
 - * Dies sind oft Backbone-Provider oder große ISPs

Jedes AS hat:

- Eine eindeutige Nummer (ASN)
- Interne Routingregeln
- Einen oder mehrere Gateways zu anderen AS

Zwei Ebenen im Routing

1. Intra-AS-Routing (innerhalb eines AS):

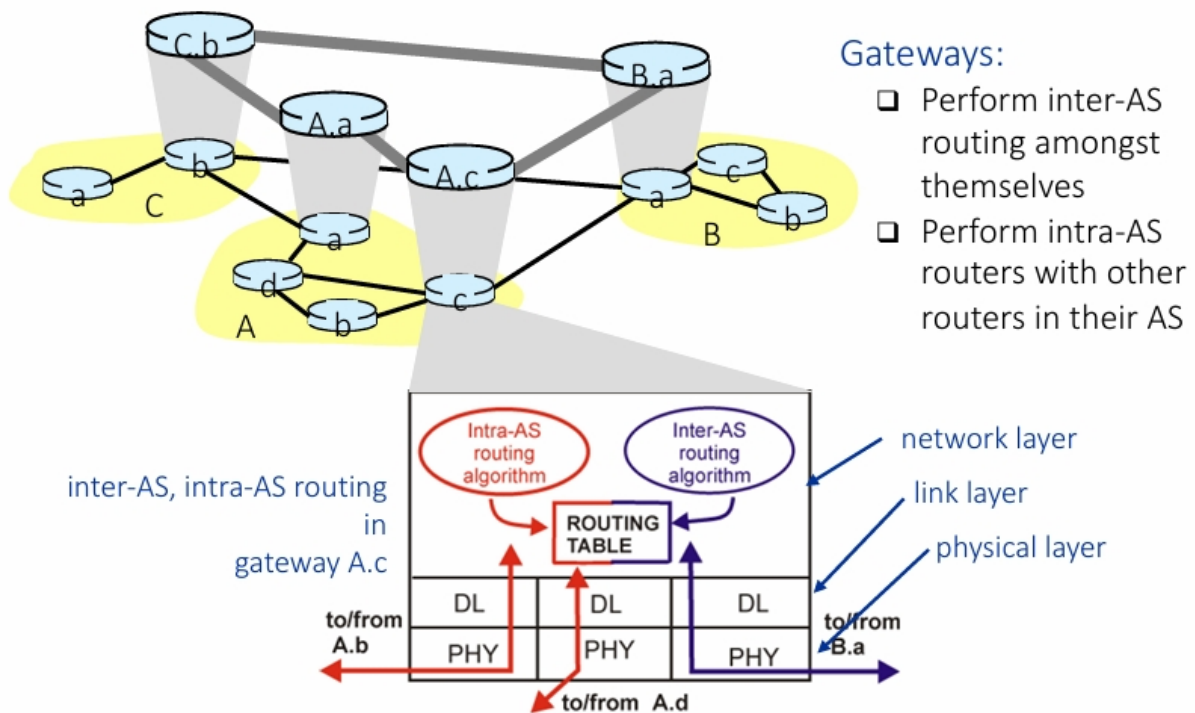
- Wird vom AS selbst kontrolliert
- Protokolle: RIP, OSPF, IGRP
- Ziele: Effizienz, schnelle Konvergenz

2. Inter-AS-Routing (zwischen AS):

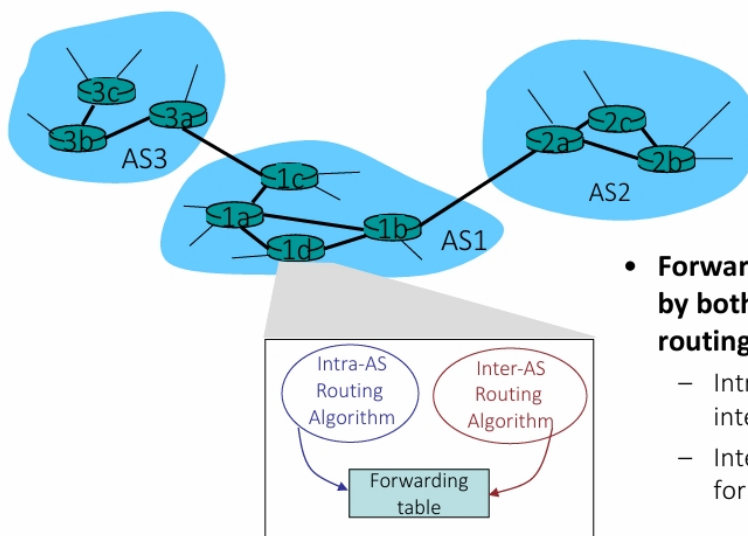
- Globaler Standard: Border Gateway Protocol (BGP)
- Enthält Pfadinformation zur Vermeidung von Loops
- Berücksichtigt auch Richtlinien („Policy Routing“)

Gateway-Router

- Sind die Brücken zwischen AS
- Sie führen sowohl Intra-AS als auch Inter-AS-Routing durch
- Empfangen Informationen über Erreichbarkeit von Subnetzen und geben diese an interne Router weiter



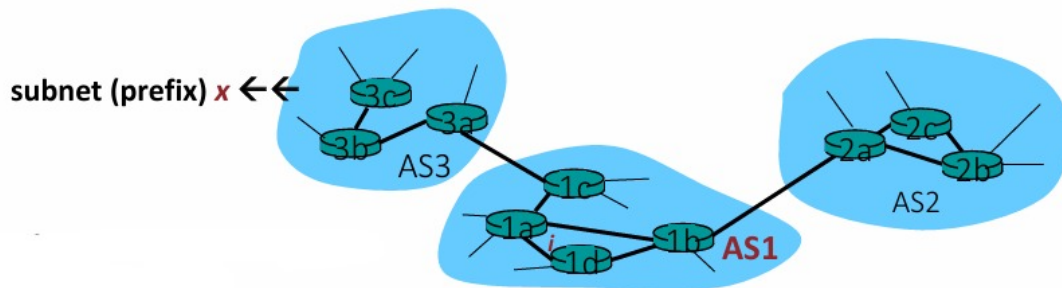
Interconnected Autonomous Systems



- **Forwarding table is configured by both intra- and inter-AS routing algorithm**
 - Intra-AS sets entries for internal dests
 - Inter-AS & Intra-AS sets entries for external dests

Beispiel: Routing über mehrere AS

1. **AS1** erfährt über BGP, dass Subnetz x über **AS2** und **AS3** erreichbar ist
2. **Router 1d** in AS1 wählt den nächstgelegenen Gateway (z.B. 1c für AS3)
3. **Hot Potato Routing**: Paket wird so schnell wie möglich an externes AS weitergegeben - unabhängig davon, ob das dann optimal ist



Warum Trennung in Inter- und Intra-AS?

Aspekt	Inter-AS	Intra-AS
Skalierung	Aggregation reduziert Tabellen	Vollständige Info über AS
Kontrolle	Richtlinien wichtig (Policies)	Ein Admin, keine Richtlinien
Performance	Zweitrangig (Policy dominiert)	Priorität auf Performance

2.7 Reflektionen über Internet-Routing

RIP, OSPF und BGP im Überblick

Protokoll	Typ	Einsatz	Eigenschaften
RIP (1983)	Distance Vector	Intra-AS	Metrik = Hop Count Einfach, aber ineffizient bei großen Netzen Verwendet „Poison Reverse“ zur Loop-Vermeidung
OSPF	Link State	Intra-AS	Öffentlich verfügbar Nutzt TCP zur zuverlässigen Übertragung Zwei Metriken: Best Effort & Realtime Multicast-fähig
BGP	Path Vector	Inter-AS	De-facto Standard für Inter-Domain-Routing Unterstützt Policies Arbeitet über dauerhafte TCP-Sessions

BGP-Grundlagen

- **BGP (Border Gateway Protocol)** ermöglicht:
 - Erhalt und Weitergabe von Erreichbarkeitsinformationen (Prefixe) zwischen AS
 - Auswahl „guter“ Routen auf Basis von Policy und Pfadinformation

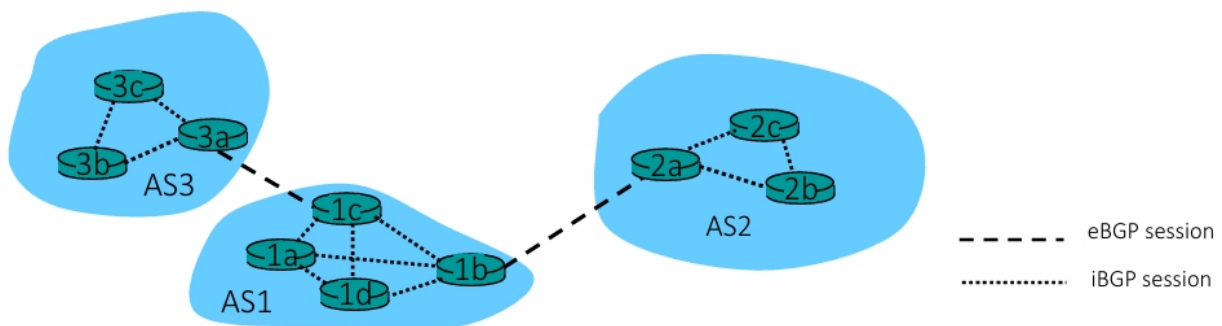
- Prefixe (z.B. Subnetze) werden angekündigt: „Ich bin erreichbar“
- Routerpaaren (BGP-Peers) tauschen Routinginformationen über **BGP-Sessions** (TCP) aus
- **Wichtig:** BGP-Sessions $\neq$ physikalische Links

BGP-Session Arten

Art	Beschreibung
eBGP (external BGP)	Zwischen zwei verschiedenen AS
iBGP (internal BGP)	Innerhalb eines AS zwischen den Routern

Verteilung von Reachability Information

1. AS3 sendet über **eBGP** Prefixe an AS1
2. Router 1c leitet Info per **iBGP** an andere AS1-Router (z.B. 1d)
3. Router 1b kann neue Info an AS2 (z.B. Router 2a) weitergeben
4. Für jedes neue Prefix wird ein neuer Eintrag in die Forwarding Table aufgenommen



BGP-Attribute = Pfadinformation + Richtlinien

- **AS-PATH:** Liste durchlaufener AS (z.B. „AS67 AS17“)
- **NEXT-HOP:** Gibt den nächsten Router in Richtung des beworbenen Prefixes an
- Wird verwendet, um Loops zu vermeiden und Routingentscheidungen zu treffen

BGP-Routing-Policies

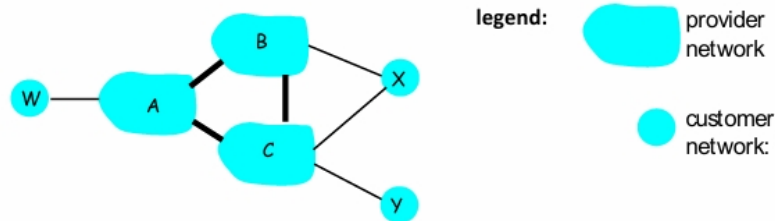
Routingentscheidungen basieren oft auf Geschäftsbeziehungen, nicht auf kürzester Router:

- **Provider-Kunde-Beziehung:**
 - Provider leitet Verkehr **von** und **zu** seinen Kunden weiter
 - Nicht erlaubt: Weiterleitung **zwischen** anderen Providern (kein Transit ohne Vertrag)

Beispiel:

- X ist Kunde von A und B (dual-homed)

- X möchte nicht als Transit zwischen B und C fungieren
- B sollte daher Route „C → B → A → W“ **nicht** weitergeben



Skalierungsprobleme

- BGP skaliert gut durch AS-Trennung, **aber**:
 - iBGP erfordert **vollständige Mesh** unter Routern
 - Mehr AS = größere Routingtabellen = langsamere Konvergenz
 - Änderungen in einem AS können viele andere AS indirekt beeinflussen

BGP-Probleme & Risiken

Kategorie	Beispielprobleme
Routinprobleme	- Suboptimale Pfade - Instabile Routen - Route Flapping
Sicherheit	- Blackholing - Traffic Hijacking - Fehlgeleitete Datenströme durch falsche Ankündigungen

2.8 Mobile Routing - Herausforderungen & Lösungen

Herausforderungen beim Routing in mobilen Netzen

- **Mobilität**: Netzwerkknoten bewegen sich → Topologie ändert sich häufig
- **Probleme**:
 - Veralterte Routing-Informationen
 - Hohe Routing-Overhead bei klassischen Protokollen (z.B. OSPF, RIP)
 - Latenz durch ständige Pfadanpassung
- **Frage**: Können klassische Routing-Algorithmen verwendet werden?
 - Eher **nein**, da sie auf statischen Topologien und periodischen Updates beruhen
- **Lösungsansatz**: Neue, **reaktive** Routing-Protokolle wie DSR

Dynamic Source Routing (DSR)

Grundprinzipien von DSR:

- **Reaktiv statt proaktiv:**
 - Kein periodisches Austauschen von Routeninformationen
 - Routing erfolgt **on-demand**
- **Zwei Hauptphasen:**
 1. **Pfadfindung (Path Discovery)**
 2. **Pfadpflege (Path Maintenance)**
- **RFC 4728** (IETF-Standard)

Source Routing:

- Der **Sender** legt den gesamten Pfad im Paket-Header fest
- Jedes Paket trägt den vollständigen Routenverlauf

Path Discovery

- Wird **nur bei Bedarf** gestartet
- Verwendet **Flooding**: Broadcast mit Zieladresse und eindeutiger ID
- Empfänger überprüft:
 - Schon gesehen? → Verwerfen
 - Ziel erreicht? → Pfad rücksenden
 - Sonst: Adresse anhängen und weiter broadcasten
- **Assumption**: Netzwerk ist wenigstens temporär verbunden
- Pfad wird im Rückpaket an den Sender übermittelt
- **Problem**: Kein garantiert bidirektionaler Pfad → Rückweg evtl. ebenfalls flooden

Path Maintenance

- Nur für **genutzte Pfade**
- **Ziel**: Sicherstellen, dass Pfad weiter funktioniert
- **Möglichkeiten zur Fehlererkennung**:
 - Layer 2 Acknowledgement (z.B. in WLANs)
 - Mithören von Weiterleitungen (sofern möglich)
 - Aktives Anfordern von ACK
- **Bei Problemen**:
 - Fehler an Sender melden oder
 - neuen Pfad vom aktuellen Zwischenknoten aus suchen

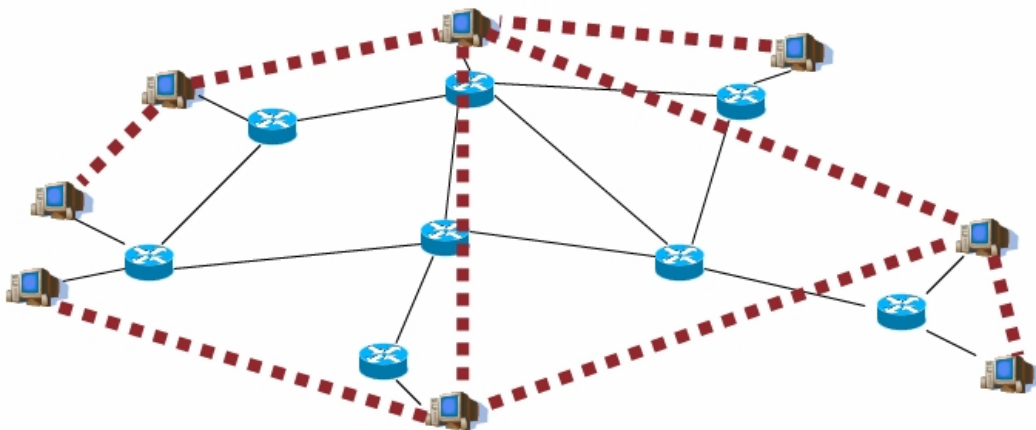
2.9 Overlay Routing

Was ist Overlay Routing?

- Dritte große Routing-Variante neben Distance Vector & Link State
- Ein **Overlay-Network** ist ein **virtuelles Netz**, das auf einem realen Netzwerk aufgesetzt ist
 - Knoten sind logisch verbunden, aber nicht unbedingt physikalisch
- Typische Beispiele:
 - Peer-to-Peer-Netzwerke (Gnutella, BitTorrent)
 - Cloud/Datacenter-Systeme (z.B. Memcached, CamCubeOS)
 - IP selbst ist ein Overlay über physischen Netzwerken

Eigenschaften von Overlay-Netzen

- **Overlay-Link $\neq$ IP-Link:** Logische Nachbarn müssen nicht direkt verbunden sein
- **Beispiel-Schichten:**
 1. Physikalische Netzwerke (Kabel, Glasfaser)
 2. IP über Physik
 3. Overlay (z.B. P2P) über IP
 4. Anwendung über Overlay

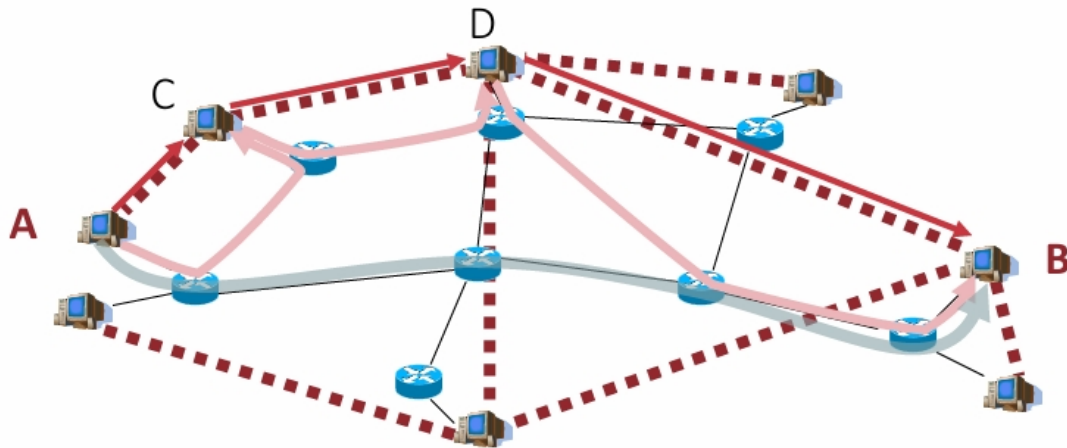


Routing in Overlay-Netzen

- Grundsätzlich: **gleiche Routing-Algorithmen** wie in IP (z.B. Dijkstra, Bellman-Ford)
- Aber zwei wichtige **Metriken**:
 1. **Overlay-Hop-Count** - Zahl der Overlay-Knoten auf dem Weg
 2. **Stretch** - Verhältnis von Overlay-Pfadlänge zur optimalen IP-Pfadlänge
⇒ Ziel: Stretch minimieren

Beispiel:

- Overlay-Pfad: $A \rightarrow C \rightarrow D \rightarrow B$ (jeweils 3, 3, 4 IP-Hops $\rightarrow$ total 10)
- Direkter IP-Pfad $A \rightarrow B = 5$ IP-Hops
 $\rightarrow$ Stretch = $10/5 = 2$



Strategien für Overlay Routing

Herausforderungen:

- Ressourcen/Nutzer unbekannt
- Knotenverfügbarkeit schwankt (Churn)
- Ressourcen sind verteilt

Lösungsstrategien:

1. Organisation:

- Flat/random (z.B. Gnutella)
- Hierarchisch
- Strukturiert (z.B. DHTs mit Schlüssel-Raum)

2. Delegationstyp:

- Rekursiv: Weiterleitung durch Overlay-Knoten
- Iterativ: Sender fragt alle selbst ab

Beispiele:

- Gnutella, eMule, BitTorrent (P2P-Dateitausch)
- Memcached (verteilte Datenbank-Caches)
- CamCube OS (Microsoft, 3D-Torus-Strukturen)

3 Internetworking + DNS

3.1 Internet Protocol (IP)

Was ist das Internet?

Das Internet ist ein Netzwerk von Netzwerken - also ein Zusammenschluss vieler kleinerer, oft sehr unterschiedlicher Netzwerke:

- **Internet = „Inter“-Netzwerk:** Ein globales System, das viele kleinere Netzwerke miteinander verbindet
- **Router als zentrale Verbindungsstellen:** Router übernehmen das Weiterleiten (Forwarding) von Datenpaketen zwischen Netzwerken
- **Kleinere Netzwerke = LANs:** Oft basieren sie auf dem IEEE-802-Standard (z.B. Ethernet, WLAN)
- **Zwei Arten der Kommunikation:**
 - **Lokal (Local):** Kommunikation innerhalb eines LANs, ohne Router
 - **Remote (Fern):** Kommunikation über Router hinweg - über das Internet

Was braucht man, damit das Internet funktioniert?

Damit verschiedene Netzwerke miteinander kommunizieren können, sind mehrere Komponenten nötig:

1. Logische Adressierung

- Physikalische Adressen (z.B. MAC-Adressen von IEEE 802) sind nicht ausreichend
- **IP-Adressen** (z.B. 192.168.1.1) dienen als logische Adressen, unabhängig von der zugrundeliegenden Hardware

2. Zuordnung von logischen zu physikalischen Adressen

- z.B. mit **ARP (Address Resolution Protocol)**
- Wird benötigt, um Pakete im lokalen Netz korrekt zuzustellen

3. Mechanismus zur Unterscheidung: lokal oder remote?

- Beim Senden eines Pakets muss entschieden werden:
 - Ist der Empfänger im selben LAN?
 - Oder muss das Paket über das Internet weitergeleitet werden?
- Diese Entscheidung trifft meist der Router anhand von Routing-Tabellen

4. Unterstützung für viele Technologien

- Das Internet muss verschiedenste Übertragungstechniken unterstützen:
 - Ethernet, WLAN, Glasfaser, Mobilfunk usw.
 - Dazu gehören auch Funktionen wie:
 - **Fragmentierung** (Aufteilen großer Pakete)
 - **Diagnose** (z.B. ICMP für Fehlermeldungen)
- All das wird durch das Internetprotokoll **IP (Internet Protocol)** möglich gemacht

IPv4 - Das klassische Internetprotokoll

- **Historie**

- Ursprünglich definiert in RFC 760 (Januar 1980)
- Weltweit im Einsatz - lange das Rückgrat des Internets
- Verwendet 32-Bit-Adressen → max. ca. 4,3 Mrd. eindeutige Adressen

- **IPv4 Adressierung**

- Beispieladresse: 192.168.1.53
- Darstellung in vier Dezimalblöcken (z.B. 192.168.0.1)

- **Helferprotokolle für IPv4**

Funktion	Protokoll
Adressvergabe	DHCP
Auflösung IP → MAC	ARP (Request/Reply)
Fehlermeldungen, Diagnose	ICMPv4
Multicast-Gruppen	IGMP

- **Subnetting**

- Aufteilung eines Netzwerks in Subnetze mithilfe von Subnetzmasken, z.B. 255.255.255.0
- Dient zur Unterscheidung von Netzanteil in der Adresse

- **Beispielkommunikation (vereinfacht):**

- Host A will Daten an 192.168.1.42 senden
- Prüft: Ist Ziel im eigenen Netz?
- → Wenn ja: ARP-Request → MAC-Adresse von 192.168.1.42 wird gefragt
- → Dann: Direkt senden. Falls nein: Routing über Standard-Gateway

Subnetzmaske

- **Was ist eine Subnetzmaske?**

Eine **Subnetzmaske** (engl. subnet mask) trennt bei einer IP-Adresse den **Netzwerkanteil** vom **Hostanteil**. Sie wird gemeinsam mit einer IP-Adresse verwendet, um zu bestimmen:

- **Zu welchem Netzwerk** eine IP-Adresse gehört
- **Welche Adressen** im selben Subnetz liegen

- **Beispiel einer IPv4-Adresse und ihrer Subnetzmaske:**

- IP-Adresse: 192.168.1.10
- Subnetzmaske: 255.255.255.0

In binärer Form:

1	IP-Adresse:	11000000.10101000.00000001.00001010
2	Subnetzmaske:	11111111.11111111.11111111.00000000

Die 1-Bits in der Subnetzmaske markieren den **Netzwerkanteil**, die 0-Bits den **Hostanteil**
In diesem Fall:

- Netzwerkanteil: 192.168.1
- Hostanteil: 10

Das bedeutet: Alle Geräte mit IP-Adressen von 192.168.1.1 bis 192.168.1.254 liegen im **gleichen Subnetz**

- **Wozu braucht man Subnetzmasken?**

1. **Netzwerke unterteilen (Subnetting)**

Große Netzwerke werden in kleinere Subnetze aufgeteilt, um:

- Broadcast-Verkehr zu reduzieren
- Sicherheit zu erhöhen
- IP-Adressen effizienter zu verwalten

2. **Routing**

Router verwenden Subnetzmasken, um zu entscheiden, ob ein Zielgerät **im eigenen Netzwerk** liegt oder ob das Paket **weitergeleitet** werden muss

- **CIDR-Notation**

Subnetzmasken werden oft in **CIDR-Notation** geschrieben, z.B.:

```
192.168.1.10/24
```

Die Zahl 24 steht für die Anzahl der 1-Bits in der Subnetzmaske:

```
/24 = 255.255.255.0
```

Einige gängige CIDR-Werte:

CIDR	Subnetzmaske	Anzahl Hosts
/8	255.0.0.0	16.777.214 Hosts
/16	255.255.0.0	65.534 Hosts
/24	255.255.255.0	254 Hosts
/30	255.255.255.252	2 Hosts

(immer abzüglich Netz- und Broadcast-Adresse)

IPv6 - Die moderne Alternative

- **Historie & Motivation**

- Entwickelt ab 1995, RFC 2460 erschien 1998
- Ziel: Rückkehr zur Einfachheit & Autonomie der Anfangsjahre
- Hauptgründe für Einführung:
 - * End-to-End Prinzip wiederherstellen
 - * Riesiger Adressraum (128 Bit → $3,4 \times 10^{38}$ Adressen)

- **IPv6 Adressierung**

- Beispieladresse: 2001:0db8::1
- Hexadezimal dargestellt
- Automatische Konfiguration möglich (SLAAC)
- Trennung in Netz- und Hostanteil meist bei /64

- **Helferprotokolle für IPv6**

Funktion	Protokoll
Adressvergabe	DHCPv6, SLAAC
Auflösung IP → MAC	NDP (NS/NA)
Routinginformation	RA, RS
Multicast-Management	MLD
Fehlermeldungen, Diagnose	ICMPv6

Warum ist der Umstieg so schwierig?

- **Vorteile von IPv6:**

- Fast unbegrenzter Adressraum
- Keine Konflikte durch private IPv4-Adressen & NAT
- Entlastung bei IPSs durch Wegfall komplizierter NAT-Strukturen

- **Nachteile bzw. Herausforderungen:**

- Nicht kompatibel mit IPv4 → beide Protokolle müssen parallel betrieben werden (Dual Stack)
- Anpassung von Netzwerk- und Sicherheitsinfrastruktur nötig
- Geringer Nutzen für viele Unternehmen → hohes Aufwand-Nutzen-Verhältnis

3.2 Domain Name Service (DNS)

Was ist das Problem, das DNS löst?

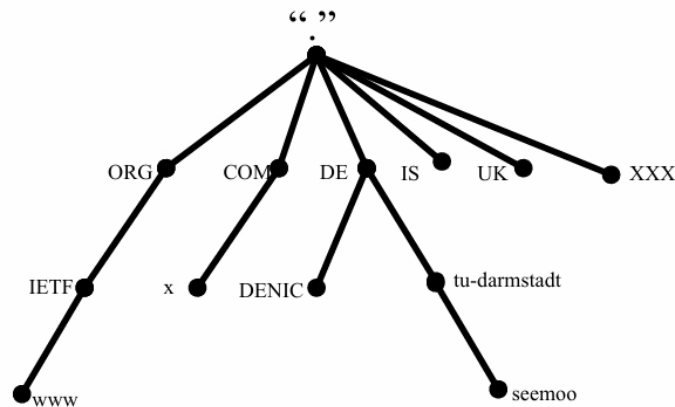
- **Ausgangspunkt**

Wir haben gelernt, dass im Internet Geräte durch IP-Adressen eindeutig adressiert werden - z.B. 130.83.47.181. Aber für Menschen ist es schwer, sich solche Adressen zu merken → Problem: Wie kann Alice Bob kontaktieren, wenn sie seine IP-Adresse nicht kennt?

- **Lösung**

Das Domain Name System (DNS) löst dieses Problem:

- DNS ist ein verteilter Dienst, der Hostnamen (wie `www.tu-darmstadt.de`) in IP-Adressen übersetzt
- Man nennt das auch „Namensauflösung“



Warum braucht man verschiedene Arten von Adressen?

Im Internet gibt es auf verschiedenen Schichten unterschiedliche Adresstypen:

Ebene	Beispiel	Art der Adresse	Eigenschaften
Application Layer	www.tu-darmstadt.de	Hostname	Menschlich lesbar, hierarchisch
Internet Layer	130.83.47.181	IP-Adresse (IPv4/IPv6)	Maschinenlesbar, hierarchisch
Data Link Layer	00:97:F1:12:34:56	MAC-Adresse	Maschinenlesbar, flach (nicht hierarchisch)

→ Deshalb braucht man Mechanismen, um zwischen diesen Adresstypen zu übersetzen (DNS für Hostname → IP, ARP für IP → MAC)

Hierarchical vs. Flat

- Hierarchisch

- Die Adressen sind **in Ebenen oder Schichten organisiert**
- Sie können in Teile zerlegt werden, die jeweils eine Bedeutung haben (z.B. bei einer IP-Adresse: Netzwerkanteil und Hostanteil)
- Das hilft, die Adressen effizient zu strukturieren, zu verwalten und zu routen
- Beispiel: IP-Adressen - der Teil links bestimmt das Netzwerk, der Teil rechts das Gerät im Netzwerk

- **Flach:**

- Die Adressen sind **eindimensional ohne Struktur oder Unterteilung**
- Jedes Gerät hat eine eindeutige, aber nicht hierarchisch gegliederte Adresse
- Es gibt keine „Untergruppen“ oder logische Schichten in der Adresse
- Beispiel: MAC-Adressen - jede Adresse ist einzigartig, aber es gibt keine Teile, die eine Struktur oder Organisation anzeigen

High Order and Low Order

1. Grundidee:

- **High Order** bezeichnet den **höherwertigen, übergeordneten** Teil eines Identifikators oder einer Adresse. Er gibt eine **breite, grobe oder übergeordnete Zuordnung** an (z.B. ein Netzwerk, eine Organisation oder ein Hersteller)
- **Low Order** bezeichnet den **niederwertigen, spezifischeren** Teil eines Identifikators. Er steht für **feinere, detailliertere oder individuelle Zuordnung** (z.B. ein einzelner Host, ein Interface oder ein Gerät innerhalb der übergeordneten Struktur)

2. Beispiel:

Identifikator	High Order (übergeordnet)	Low Order (fein)
Hostname	Domain, die von einer Registrierungsstelle vergeben wird, z.B. .com, .edu oder tu-darmstadt.de (z.B. bei seemoo.tu-darmstadt.de: .de und tu-darmstadt sind High Order)	Lokaler Hostname, vom Administrator vergeben, z.B. www, cms-sip02 („www“ ist Low Order, da spezifischer Host in der Domain)
IP-Adresse	Netzwerkteil (Präfix), vergeben von ICANN, Regionalen Internetregistern (RIR) oder ISPs (z.B. bei 130.83.47.181 ist 130.83 das Netzwerk)	Host-Teil der IP-Adresse, einem Gerät im Netzwerk zugewiesen (z.B. per DHCP oder manuell) (z.B. 47.181 ist Low Order)
MAC-Adresse	Herstellerkennung (Organizationally Unique Identifier, OUI), 24 Bits von IEEE an Hersteller vergeben (z.B. 00-15-C5 gehört z.B. zu IBM, Apple)	Gerätespezifische Schnittstellenkennung, 24 Bits, vom Hersteller vergeben (z.B. 49-04-A9 ist Low Order für die Netzwerkkarte)

ICANN (Internet Corporation for Assigned Names and Numbers)

- **ICANN** ist eine **nicht-kommerzielle Organisation**, die weltweit zentrale Aufgaben für das Funktionieren des Internets übernimmt
- **Hauptaufgaben von ICANN:**
 - 1. Domain Name System (DNS) verwalten:**
ICANN koordiniert die Vergabe von Domainnamen (z.B. .com, .org, .de) und überwacht die sogenannten **Top-Level-Domains (TLDs)**
 - 2. Zuteilung von IP-Adressen:**
Sie delegiert IP-Adressblöcke anregionale Organisationen (wie RIPE für Europa), die diese weiter an Provider und Unternehmen vergeben
 - 3. Verwaltung von Root-Servern:**
ICANN verwaletet zusammen mit anderen Organisationen das Root-Server-System des Internets, das die Grundlage für DNS-Anfragen ist

4. **Zuständigkeit für Protokollparameter:**

In Zusammenarbeit mit der **IETF (Internet Engineering Task Force)** sorgt ICANN für die Koordination technischer Standards und Protokollnummern

RIR (Regional Internet Registry)

- Ein RIR ist eine Organisation, die im Auftrag von **ICANN** IP-Adressen und autonome Systemnummern (**ASNs**) in einer bestimmten geographischen Region verwaltet und zuteilt
- **Die 5 RIRs weltweit:**

Region	RIR-Name
Europa, Naher Osten, Zentralasien	RIPE NCC
Nordamerika	ARIN
Lateinamerika & Karibik	LACNIC
Afrika	AFRINIC
Asien-Pazifik	APNIC

- **Hauptaufgaben eines RIR:**
 1. **Zuteilung von IP-Adressen** (IPv4 & IPv6) und **AS-Nummern** an Internetdienstanbieter (ISPs), Unternehmen, Universitäten usw.
 2. **Führen öffentlicher Datenbanken**, um festzuhalten, wem welche IP-Adressen gehören (z.B. für Whois-Abfragen)
 3. **Förderung technischer Standards**, Schulungen, Sicherheit und verantwortungsvoll IP-Nutzung
 4. **Koordination mit ICANN** und anderen RIRs, um globale Adresspolitik umzusetzen
- **Beispiel:**

Wenn z.B. ein Internetprovider in Deutschland neue IP-Adressen braucht, beantragt er sie bei **RIPE NCC**

Implementing DNS: Alte Methode (Hosts-Datei)

- **Was:**

Eine lokale Datei (/etc/hosts auf Unix, C : \... \hosts auf Windows) enthält Zuordnungen von Hostnamen zu IP-Adressen
- **Wie:**

Bevor DNS befragt wird, prüft das System diese Datei - findet es einen Eintrag, wird die IP direkt verwendet
- **Früher:**

War notwendig, als das Internet noch klein war. Alle Hosts mussten ihre Datei manuell aktuell halten
- **Warum heute ungeeignet:**
 - Nicht skalierbar (zu viele Hosts)
 - Keine automatische Aktualisierung
 - Sicherheitsrisiken (manipulierbar)
 - Fehleranfällig
- **Heute genutzt für:**
 - Lokale Tests

- Webseiten blockieren
- Entwicklung ohne DNS

Warum kein zentrales DNS?

- DNS ist absichtlich dezentral aufgebaut:
 - Skalierbarkeit: Ein zentrales System wäre überlastet
 - Ausfallsicherheit: Single Point of Failure (SPOF) wird vermieden
 - Verwaltungshoheit: Jede Organisation kann eigene DNS-Einträge verwalten

Welche Arten von DNS-Einträge (Resource Records) gibt es?

Typ	Beschreibung
A	IPv4-Adresse
AAAA	IPv6-Adresse
CNAME	Alias auf anderen Hostnamen
MX	Mailserver (Mail eXchanger)
NS	Nameserver für eine Domain
TXT	Freitext - heute z.B. für SPF, DKIM etc.
SOA	Informationen über Zone-Verwaltung

Aufbau eines DNS Resource Records (RR):

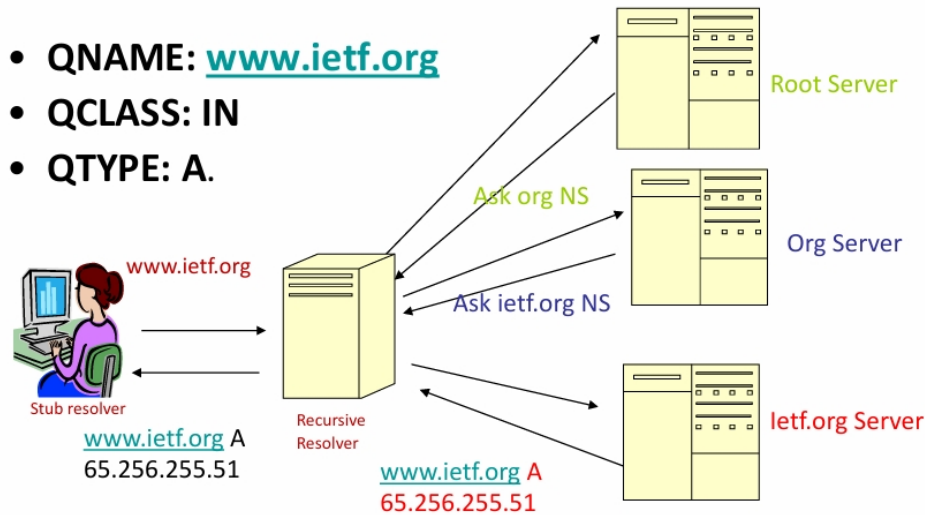
```
+-----+-----+-----+-----+-----+-----+
+ Domain name | type | class | TTL   | RL  | RDATA   |
+-----+-----+-----+-----+-----+-----+
          <variable>      2       2       4       2   <variable>
```

- **TTL:** Time To Live in Sekunden, wie lange der RR im Cache bleibt
- **Class:** Klasse des RR, meist IN für Internet
- **RL: RD LENGTH:** Länge der darauffolgenden RDATA in Bytes
- **RDATA:** Daten des RR, z.B. IP-Adresse für A, Mailserver für MX

Wie funktioniert eine DNS-Anfrage?

1. Du gibst eine Domain im Browser ein
2. Der **Stub Resolver** deines Betriebssystems schickt die Anfrage an den **rekursiven Resolver**
3. Der rekursive Resolver sucht die IP-Adresse, indem er verschiedene DNS-Server abfragt
4. Er holt sich die Antwort vom **autoritativen DNS-Server** der jeweiligen Domain
5. Der rekursive Resolver speichert die Antwort im Cache für spätere Anfragen

6. Die IP-Adresse wird an den Stub Resolver zurückgegeben
7. Dein Computer kann die Webseite jetzt mit der IP-Adresse erreichen



A: QTYPE: AAAA added for IPv6!

DNS und Sicherheit

- **DNS ist essentiell** für das Internet, weil es Domain-Namen in IP-Adressen übersetzt. Ohne DNS funktionieren viele Dienste nicht
- Ein Ausfall von DNS (z.B. durch falsches Routing wie bei Facebook 2021) kann ganze Webseiten lahmlegen
- DNS betrifft wichtige Sicherheitsaspekte:
 - **Vertraulichkeit:** DNS-Anfragen verraten, welche Seiten Nutzer besuchen
 - **Integrität:** Manipulierte DNS-Daten können Nutzer zu falschen Seiten umleiten
 - **Verfügbarkeit:** Wenn DNS nicht funktioniert oder blockiert wird, sind Webseiten nicht erreichbar
- DNS ist oft ungeschützt und anfällig für Überwachung, Manipulation und Zensur

4 Transport Layer

4.1 Connection Control

Grundidee

In Netzwerken gibt es **zwei Arten von Diensten**:

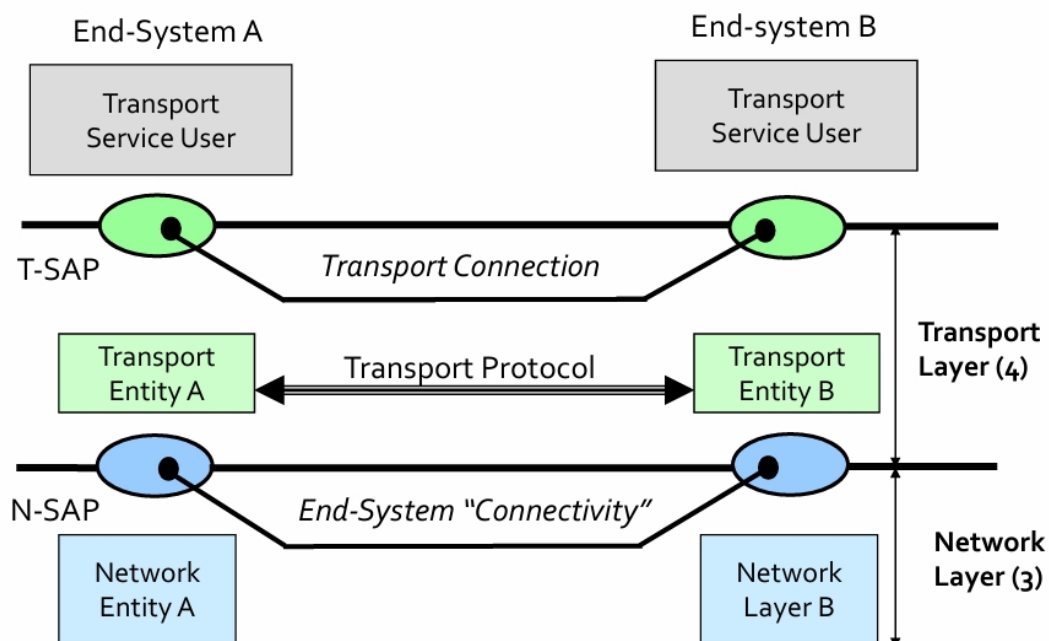
Art	Erklärung	Besipiel (Internet)
Connection-oriented (COTS)	Verbindungsaufbau vor dem Datentransfer	TCP (z.B. bei Web, E-Mail)
Connectionless (CLNS)	Direkter Versand ohne Verbindung	UDP (z.B. bei VoIP, DNS)

Im folgenden Fokus: **Connection-oriented Transport Services (COTS)**, wie sie z.B. bei TCP verwendet werden

Drei Phasen einer Verbindung (OSI-Terminologie)

Phase	Beschreibung	OSI Primitive (Beispiel)
1. Connect (Aufbau)	Eine Verbindung wird hergestellt	T-Connect.Request, T-Connect.Indication, T-Connect.Response, T-Connect.Confirmation
2. Data (Transofer)	Daten werden zwischen Endsyst- temen übertragen	T-Data.Request, T-Data.Indication
3. Disconnect (Abbau)	Die Verbindung wird ordentlich beendet	T-Disconnect.Request, T-Disconnect.Indication

Transport-Verbindung: Beteiligte

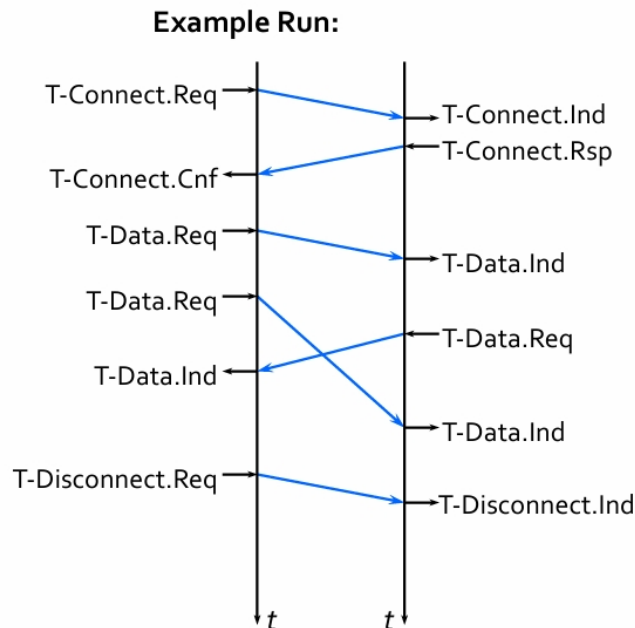


Ein **SAP (Service Access Point)** ist ein **logischer Zugangspunkt**, über den eine Schicht **einen Dienst** von der **darunterliegenden Schicht** anfordert

Service-Primitives (OSI-Sicht)

Diese sind **abstrakte Befehle**, die zwischen Schichten ausgetauscht werden, um Dienste zu nutzen

Phase 1: Verbindung aufbauen (T-Connect)



1. T-Connect.Request

- Wer? → Client (aktive Seite)
- Richtung → vom Transport Service User an die Transport-Schicht
- Bedeutung:
 - Der Client sagt: „Ich möchte eine Verbindung zu einem bestimmten Ziel aufbauen“
- **Parameter:**
 - Destination Address: z.B. IP-Adresse + Port des Servers (Zielanwendung)
 - Source Address: z.B. IP-Adresse + Port des Clients (eigene Anwendung)

2. T-Connect.Indication

- Wer? → Server (passive Seite), ausgelöst durch Transport-Schicht
- Richtung → von Transport-Schicht an die Server-Anwendung
- Die Transport-Schicht informiert die Server-Anwendung: „Hey, da möchte jemand mit dir eine Verbindung aufbauen!“
- **Wichtig:**
 - **Asynchron** - der Server „lauscht“ mit einem listen()-Mechanismus
 - Das passiert nicht durch aktiven Aufruf des Servers, sondern als **Indication (Meldung)**

3. T-Connect.Response

- Wer? → Server (passive Seite)
- Richtung → von Server-Anwendung an Transport-Schicht
- Bedeutung:
 - Der Server sagt: „Ja, ich akzeptiere die Verbindung“
- Parameter:
 - Responding Address: Adresse des Servers (kann benutzt werden, um Verbindung zu bestätigen und zu identifizieren)

4. T-Connect.Confirmation

- Wer? → Client (aktive Seite), ausgelöst durch Transport-Schicht
- Richtung → von Transport-Schicht an die Client-Anwendung
- Bedeutung:
 - Die Transport-Schicht bestätigt dem Client: „Verbindung ist aufgebaut“
- Optionaler Parameter:
 - Responding Address: Adresse des Servers, um finale Details zu bestätigen

Datenübertragung - T-Data

- **Merkmale:**
 - **Unconfirmed service:** Es wird **kein explizites Bestätigungs-Primitive (ACK)** auf der Service-Ebene gesendet
 - Trotzdem: **Das Protokoll (z.B. TCP) sendet ACKs intern!**
- **Primitives:**
 - T-Data.Request (userdata) - vom Sender aufgerufen
 - T-Data.Indication (userdata) - beim Empfänger ausgelöst

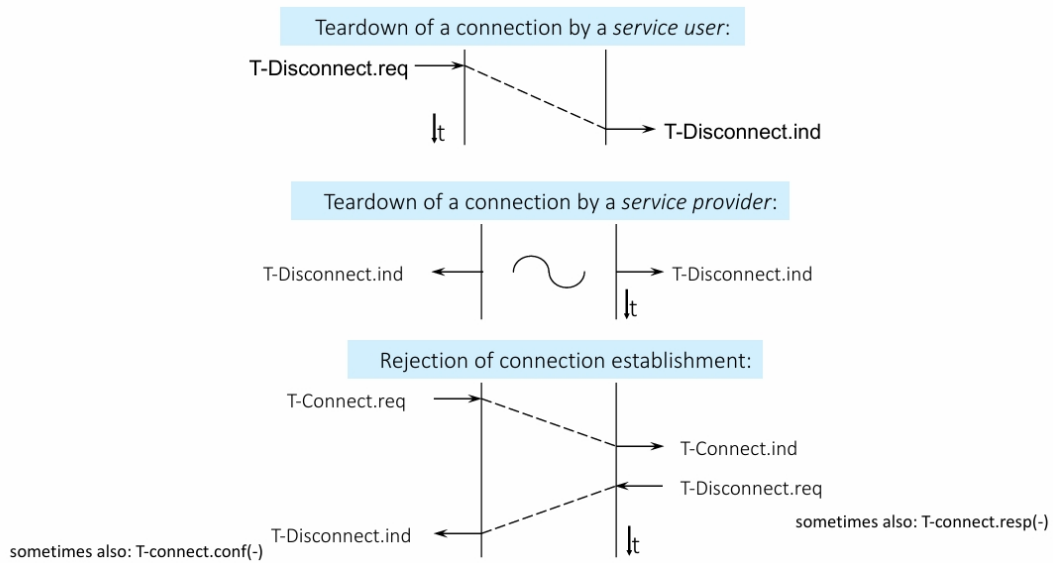
Verbindungsabbau - T-Disconnect

Wieder ein 2-Wege-Prozess:

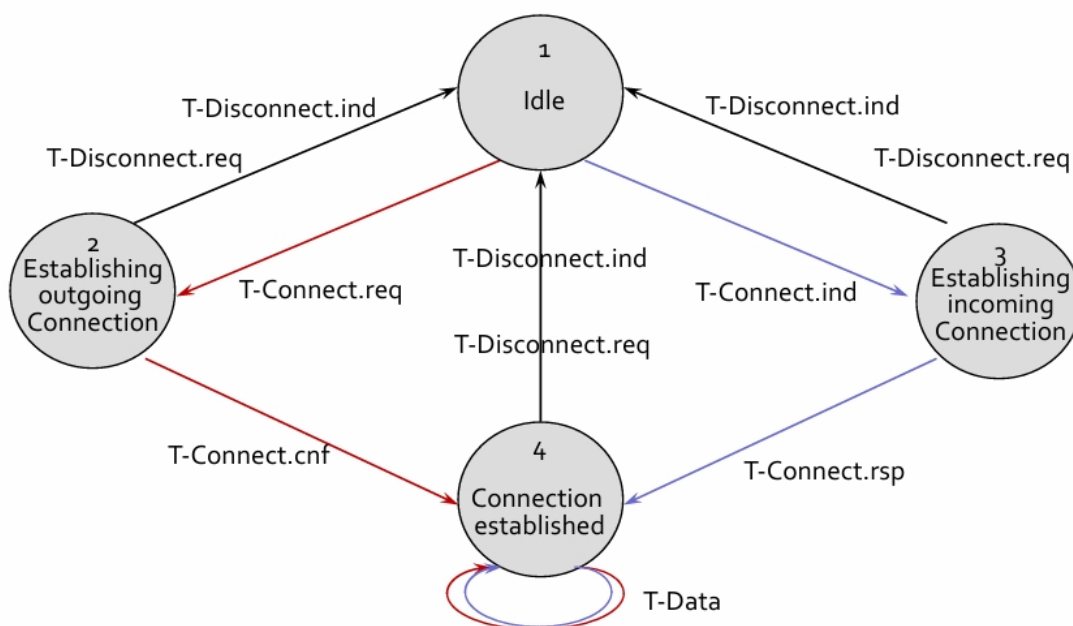
Client: T-Disconnect.Request
Server: T-Disconnect.Indication

Das entspricht dem TCP FIN/ACK-Mechanismus, also:

- Ein Ende sendet FIN
- Das andere bestätigt mit ACK



Zustandsdiagramm für einen Transport Service Access Point



Beispiel: TCP-Fall durchspielen (vereinfacht)

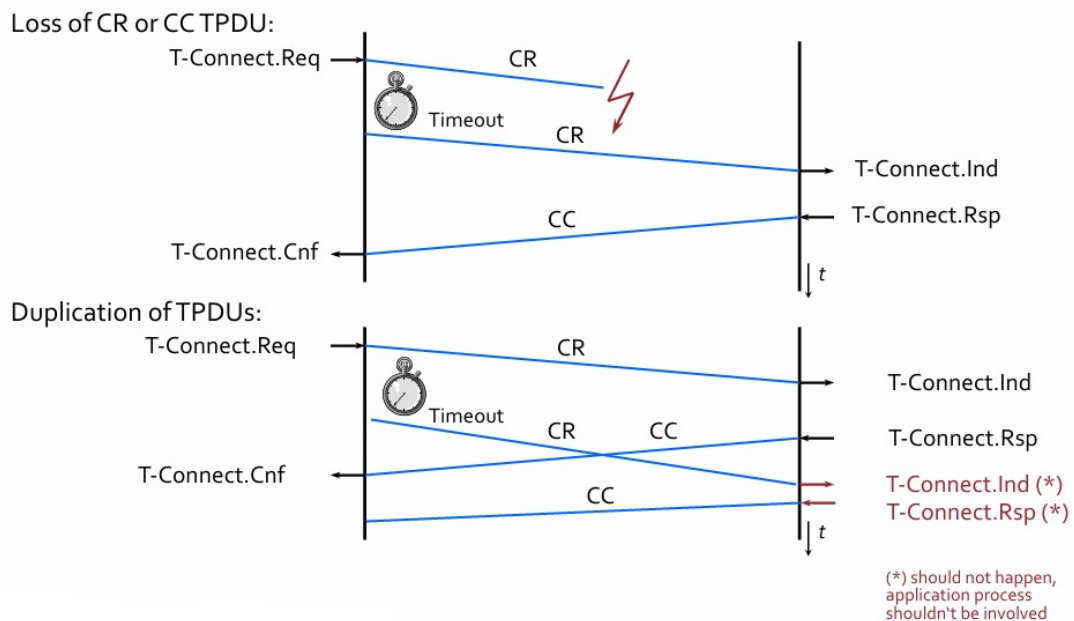
Phrase	Zustand	Service Primitive	Bemerkung
Client startet	1 → 2	T-Connect.req	TCP sendet SYN
Server erhält Anfrage	1 → 3	T-Connect.ind	TCP empfängt SYN
Server antwortet	3 → 4	T-Connect.rsp	TCP sendet SYN+ACK
Client bestätigt	2 → 4	T-Connect.cnf	TCP empfängt SYN+ACK + sendet ACK
Beide senden Daten	4	T-Data.req/ind	TCP-Transfer läuft
Verbindung wird beendet	4 → 1	T-Disconnect.req/ind	TCP sendet FIN, bekommt FIN

Errors during Connection Establishment

Beim Aufbau einer Verbindung (Phrase: Connection Establishment) werden zwischen zwei Endpunkten **Kontrollnachrichten** (TPDUs - Transport Protocol Data Units) ausgetauscht. Die wichtigsten:

- **CR** = Connection Request → vom Client gesendet
- **CC** = Connection Confirm → vom Server gesendet

Wenn dabei Fehler passieren (z.B. Nachrichten gehen verloren oder werden doppelt empfangen), kann es zu **Problemen im Verbindungsaufbau** kommen



Fehlersituationen im Detail

1. Verlust von CR oder CC (Nachricht kommt nie an)

- T-Connect.req → CR gesendet
- **CR kommt beim Server nicht an → Server reagiert nicht**
- Lösung: **Client startet nach Timeout neu**

2. Deppelte TPDUs (z.B. durch Netzwerkprobleme)

- CR oder CC wird **mehrmals empfangen**, z.B. durch Wiederholung nach Timeout

Beispiel:

- CR wird vom Client gesendet
- Server empfängt CR → sendet CC
- CC kommt beim Client nicht an → Client **wiederholt** CR
- Server empfängt CR **ein zweites Mal** → **nochmals** T-connect. ind **an Anwendung**

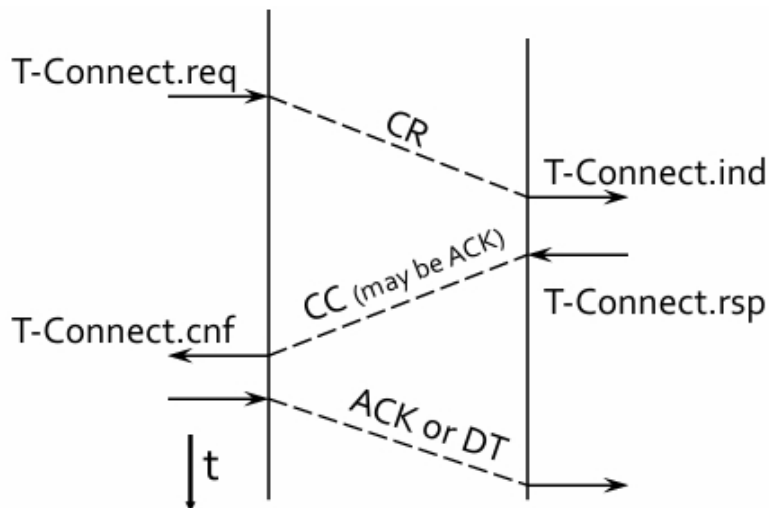
Problem: Anwendung bekommt zweimal die gleiche Verbindungsanfrage! Das darf **nicht passieren**

Die Anwendungsschicht **soll vor diesen Fehlern geschützt** werden. Solche Duplikate sollen vom **Transportprotokoll abgefangen** werden, **bevor** sie zu Anwendung gelangen

Three-Way Handshake

- **Zweck:**
 - Sicherer Verbindungsaufbau im Transport Layer (z.B. TCP)
 - Verhindert, dass Verbindungen inkonsistent oder fehlerhaft aufgebaut werden
- **Ablauf:**
 1. **CR** (Connection Request) vom Client → Server
 2. **CC** (Connection Confirm) vom Server → Client
 3. **ACK** oder erste **DT** (Data Transfer) vom Client → Server

Verbindung ist erst nach allen 3 Schritten offiziell etabliert



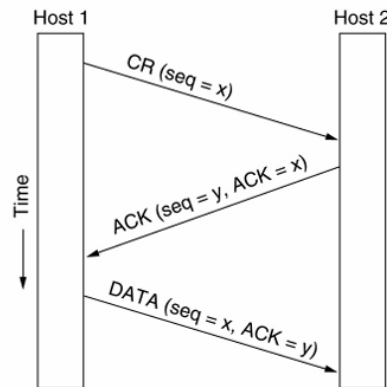
Problem: Alte oder doppelte Pakete

- Netzwerk kann alte Verbindungsanfragen (CR/CC) später erneut zustellen (sog. Replay-Angriffe)
- Gefahr: eine „vergangene“ Verbindung wird fälschlich als neu interpretiert

Lösung: Sequence Numbers

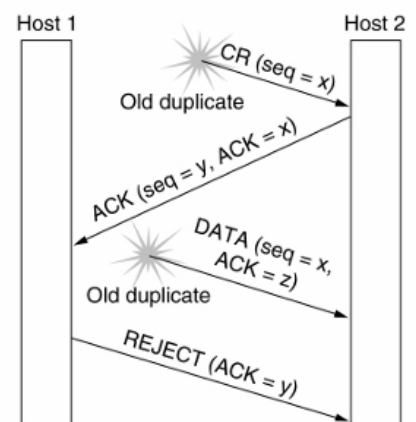
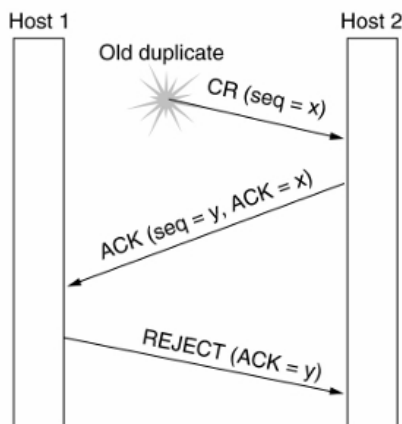
- Jeder Teilnehmer sendet **eine zufällige Nummer (SeqNum)** in CR/CC
- Die Gegenseite muss diese Nummer korrekt zurückspeiegeln
- Verbindung wird nur akzeptiert, wenn die SeqNums **zeitlich gültig und korrekt** sind

Verhindert den Aufbau auf Basis alter, duplizierter Nachrichten



Zusätzlicher Schutz

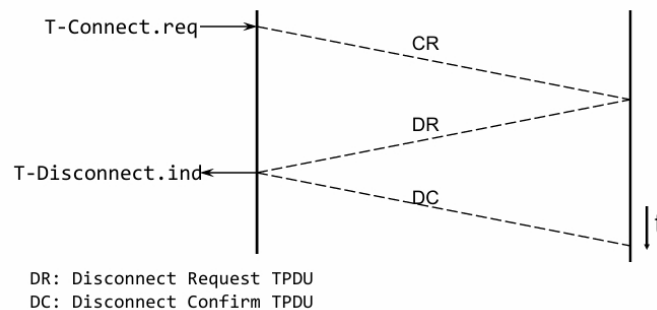
- **Sequence Numbers dürfen sich nicht wiederholen**, solange alte Pakete noch im Netz sein könnten
 - $\rightarrow$ Zeit > „Packet Lifetime“ (oft durch TTL im IP-Layer begrenzt)
- Bei sehr schnellen Verbindungen (z.B. Gbps):
 - $\rightarrow$ **Timestamps** zusätzlich verwenden
- Connection request appears as an old duplicate:
- Connection request & confirmation appear as old duplicates:



Connection Rejection (Verbindungsablehnung)

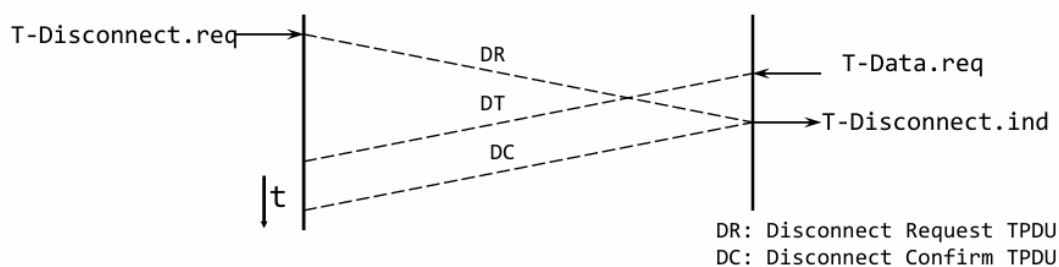
Wenn ein Transportdienst **eine eingehende Verbindungsanfrage (CR) nicht akzeptieren** will, wird sie **aktiv zurückgewiesen**

- **Wie funktioniert das?**
 - **Client sendet:** T-Connect.req (Verbindungsaufbau starten)
 - **Server lehnt ab:** mit T-Disconnect.ind und einem **DR** (Disconnect Request TPDU)
 - Danach folgt evtl. ein **DC** (Disconnect Confirm TPDU) zur Bestätigung
- **Mögliche Gründe für Ablehnung:**
 - Transportdienstnutzer will Verbindung nicht (z.B. Anwendung sagt „Nein“)
 - Dienstanforderungen (z.B. QoS) können nicht erfüllt werden



Connection Release (Verbindungsabbau)

- **Ziel:**
 - Verbindung **kontrolliert beenden**, nachdem **alle Daten sicher übertragen werden**
- **Empfohlener Ablauf:**
 1. Warten, bis alle gesendeten Daten bestätigt (ACKed) wurden
 2. Dann erst: Verbindung schließen
- **Varianten:**
 - **Explizit:** durch **DR/DC** (Disconnect Request + Confirm TPDU)
 - **Implizit:** Einfach durch Schließen der Verbindung im darunterliegenden Netzwerk



Problem beim Verbindungsabbau

Wie kannst du dir sicher sein, dass dein Kommunikationspartner **weiß**, dass **du weißt**, dass **er weiß**, dass alles angekommen ist?

→ Genau das ist das **Two-Army-Problem!**

Two-Army-Problem-Analogie

- Zwei Armeen wollen gleichzeitig angreifen, sonst verlieren sie
- Sie schicken Boten zueinander - aber Boten können **verloren** gehen
- Sie wissen nie, ob die Nachricht **wirklich angekommen** ist
- Ergebnis: **keine sichere Koordination möglich**, wenn Nachrichten verloren gehen können
- **Gleiches Problem beim Connection Release!**
- Transport Layer kann **nicht sicher wissen**, ob wirklich beide Seiten bereit sind zu schließen



Praktische Lösung: 3-Way Handshake für Verbindungsabbau

1. Peer A sendet **DR**
2. Peer B antwortet auch mit **DR** (ggf. inkl. ACK)
3. A bestätigt, Verbindung kann geschlossen werden

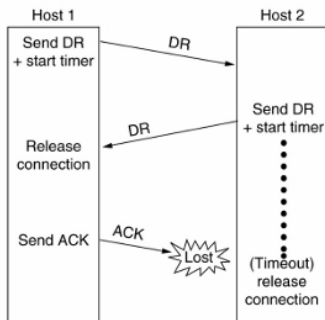
→ Ist zwar **100% sicher**, aber:

- In der Praxis funktioniert das **gut genug**
- **Timer** und **Wiederholungen** helfen bei verlorenen DR/ACK

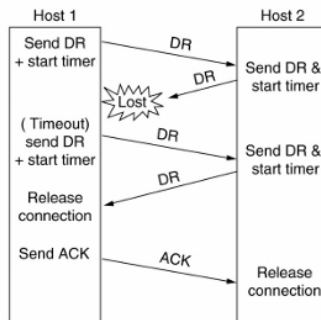
Zusätzliche Problemfälle:

- **ACK verloren:** Timer läuft aus → erneutes DR senden
- **2. DR verloren:** ebenfalls durch Timer abgefangen
- Wenn **beide nachrichten verloren:** Verbindung wird trotzdem nach Timeout abgebrochen

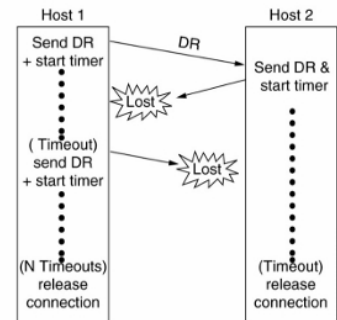
- **Lost ACK solved by (optimistic) timer in Host 2**



- **Lost second DR solved by retransmission of first DR**



- **Timer solves (optimistically) case when 2nd DR and ACK are lost**



4.2 Flow Control

Flow Control bedeutet, dass ein **Sender daran gehindert wird**, zu viele Daten zu schnell an einen **langsamen Empfänger** zu senden. Ziel: Den **Empfänger vor Überlastung** zu schützen!

Hintergrundproblem: Zuverlässige Übertragung ist schwer

Auch wenn Pakete zuverlässig zugestellt werden sollen, können viele Dinge schief laufen:

- Pakete gehen verloren (z.B. durch Überlastung oder Bitfehler)
- Pakete kommen fehlerhaft an
- Der Empfänger verarbeitet Daten langsamer, als sie ankommen

Lösungsidee: ACKs und Retransmission

Damit der Sender weiß, ob Daten korrekt ankamen:

- Der Empfänger schickt ein **ACK** (Bestätigung) für jedes empfangene Paket
- Kommt kein ACK → Sender **sendet Paket erneut** (Der Sender muss eine Kopie des Pakets behalten, bis er das ACK sicher erhalten hat - nur dann kann er es löschen)

Aber: Wie lange soll der Sender ein Paket speichern?

- Solange, bis er ein ACK erhält
- Das braucht Speicher (= **Buffer**) → Also **nicht unbegrenzt machbar**

Flow Control - Warum wichtig?

- **Problem:** Ein **schneller Sender** kann einen **langsamen Empfänger** überfordern, wenn er **ungebremst Daten schickt**
- **Ziel:** Empfänger muss **Sender bremsen können**, wenn sein **Eingangspuffer (Buffer)** voll ist

Wo wird Flow Control gemacht?

- **Link Layer:** begrenzt Datenrate zwischen zwei direkt verbundenen Geräten
- **Transport Layer:** steuert den Datenfluss **zwischen Anwendungen** auf verschiedenen Rechnern
 - Komplizierter, da:
 - * TL-Pakete unterschiedlich groß sind
 - * Anwendungen (z.B. Webserver) sehr unterschiedlich reagieren
 - * Mehrere parallele Verbindungen gleichzeitig laufen können

Buffer Management

Sender und Empfänger müssen mit **Pufferplätzen** umgehen:

- Der **Empfänger** braucht Platz, um Pakete zwischenspeichern, bevor er sie verarbeitet
- Der **Sender** muss wissen, **wie viel Platz beim Empfänger verfügbar ist**

Zwei Optionen:

1. **Optimistische Annahme:** Empfänger verarbeitet Pakete sofort → unrealistisch
2. **Guthabensystem (Credit-Based):**
 - Empfänger sagt: „Ich habe noch **X freie Buffer**“
 - Sender darf **nur so viele neue Pakete schicken**, wie **Credits** vorhanden sind→ So kann der Empfänger den **Datenfluss kontrollieren**, ohne überlastet zu werden

Wie kommuniziert der Empfänger seine Kapazität?

- **Explizit:** „Ich habe gerade 4 freie Plätze“
- **Implizit:** Durch Ausbleiben von ACKs oder bestimmte TCP-Felder

4.2.1 Alternating Bit Protocol (ABP)

Das **ABP** ist ein einfaches Protokoll zur **sicheren (zuverlässigen) und geordneten Datenübertragung** über ein unsicheres Netzwerk - also ein **Stop-and-Wait-Protokoll mit Fehlererkennung**

Grundidee

1. Der **Sender** sendet **ein Paket mit einer Bit-Nummer** (entweder 0 oder 1)
2. Der **Empfänger** erhält das Paket und schickt ein **ACK mit der gleichen Bit-Nummer** zurück
3. Der Sender wartet auf das ACK, **bevor** er das nächste Paket sendet - das nächste mit **wechselndem Bit** ($0 \leftrightarrow 1$)

Wenn kein ACK rechtzeitig kommt (Timeout), **wird dasselbe Paket nochmal gesendet**

Warum nur 1 Bit?

Weil immer **nur ein Paket gleichzeitig** unterwegs ist. Das Bit dient zur Erkennung von **Doppelübertragungen**

Situation	Erklärung
Paket kommt neu → Bitnummer neu	→ Empfänger verarbeitet & bestätigt
Paket kommt doppelt → Bitnummer wie beim letzten Mal	→ Empfänger verarbeitet es nicht nochmal und schickt ACK (Bit = Bitnummer)

Ist ABP effizient?

Leider **nein** - und das ist ein zentrales Lernziel:



- **Entfernung:** New York ↔ Frankfurt = 6.200 km
- **Lichtgeschwindigkeit in Glasfaser:** 300.000 km/s
- **Propagation through the medium:**

$$T_{prop} = \frac{6200km}{300000 \frac{km}{s}} = 20ms$$

- **Transmission Time:**

$$T_{trans} = \frac{8KBit}{10^9 \frac{Bit}{s}} = 8\mu s \text{ (assume } 1 \mu s \text{ for ACK)}$$

- **Delay until ACK of packet received:** 40009 ms
- **Utilization of medium:**

$$U_{\text{Sender}} = \frac{0,008}{40,009} = 0.0002$$

- And effectively around 25 kByte/s (in a 1 GBit/s medium)

Also:

- Extrem ineffizient auf langen Strecken und bei hoher Bandbreite
- Aber einfach und sicher für kurze Distanzen (z.B. Embedded-Bereich)

Konzept	Was passiert?	Vorteil	Nachteil
Stop/Continue Flow Control	Empfänger schickt Stop/-Continue Nachrichten	Einfach zu verstehen	Verzögerung kann Overflow verursachen
Rate-based Flow Control	Empfänger gibt max. Datenrate an	Geringer Overhead, „smooth“	Kein Empfangs-ACK, Fehler schwierig
Credit-Based Flow Control	Empfänger gibt explizites Guthaben (Pakete senden erlaubt)	Effizient, kombiniert mit ACKs	Komplexer, benötigt Fehlerkontrolle
Sliding Window	Sender darf mehrere Pakete im Fenster senden	Sehr effizient, verbreitet	Verwaltung von Fenstergröße nötig

4.3 Error Control

Problem: Nicht alle Übertragungsfehler werden erkannt

- Bei Datenübertragungen können **Bitfehler** auftreten (z.B. durch elektromagnetische Störungen)
- Man verwendet **Prüfsummen (Checksums)**, um solche Fehler zu erkennen
- Aber: **Es gibt immer ein Restrisiko**, dass ein Fehler **nicht entdeckt wird**, wenn die falschen Daten zufällig dieselbe Prüfsumme haben ⇒ **residual error**

Verhalten auf Empfängerseite bei erkannten Fehlern:

- Wenn ein Fehler erkannt wird (z.B. durch Prüfsumme in Layer 2 oder Layer 4)
 - In den meisten Protokollen reagiert der Empfänger **NICHT** aktiv
 - Das fehlerhafte Paket wird **einfach verworfen (drop)**
- Warum wird einfach verworfen?
 - Gefahr: **Ein fehlerhaftes Headerfeld** (z.B. Zieladresse) könnte das Paket **in eine falsche Richtung** weiterleiten ⇒ **besser verwerfen**
 - Es gibt **meist keine getrennte Prüfsumme für Header und Nutzdaten**, deshalb ist der Header nicht separat überprüfbar

Besser wäre: Empfänger meldet Fehler zurück → Negative ACK (NAK)

- Beispiel: „Ich habe ein fehlerhaftes oder fehlendes Paket erkannt, bitte sende es erneut“
- Vorteil:
 - Spart Zeit, weil der Sender **nicht auf Timeout warten muss**
 - Besonders bei **verzögerungsanfälligen Netzwerken (Internet)** sinnvoll
- Aber ist das möglich, wenn der Empfänger das Paket einfach verwirft
 - Ja, **indirekt**:
 - * Wenn das **nächste Paket** empfangen wird, erkennt der Empfänger:
 - „Moment, das Paket ist **außer der Reihenfolge!**“ ⇒ Fehler erkannt!
 - So kann er einen **NAK** senden oder auf das verlorene Paket hinweisen

Reihenfolgennummern sind unerlässlich

- Um:
 - **Verlorene Pakete zu erkennen**
 - Pakete **korrekt zuzuordnen**
 - Daten beim Empfänger wieder **in richtige Reihenfolge zu bringen**
 - In **ACKs/NAKs referenzieren** zu können
- Besonders kritisch bei **IP**, da Pakete **nicht zwingend in Reihenfolge** ankommen

Letzter Ausweg: Timeout beim Sender

- Wenn keine Bestätigung für ein Paket kommt ⇒ **Sender wartet Timeout ab** und sendet **erneut**
- Wichtig:
 - Timeout-Dauer ist entscheidend: **zu kurz** → **unnötige Wiederholungen**, **zu lang** → **unnötige Verzögerungen**

4.3.1 Automatic Repeat reQuest (ARQ)

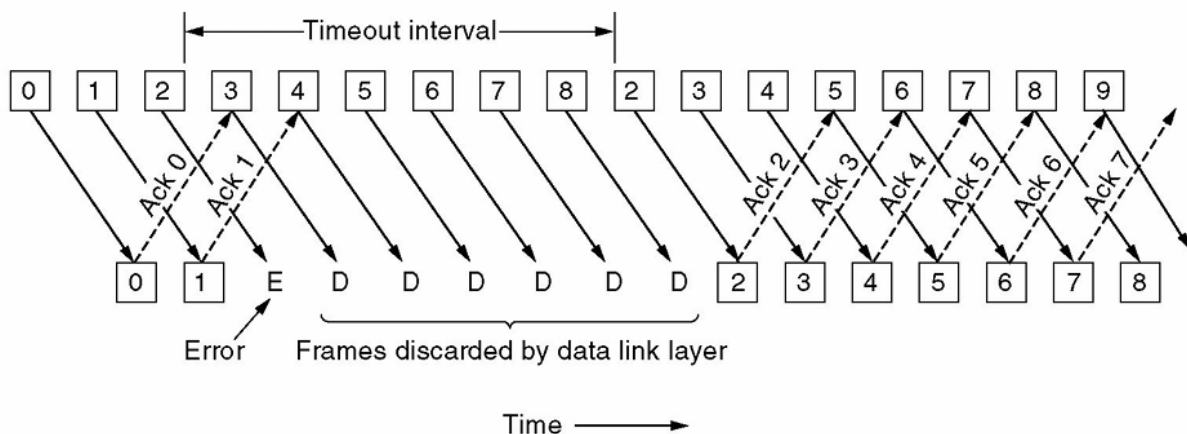
ARQ = Protokollmechanismus, bei dem der **Sender verlorene oder fehlerhafte Pakete erneut sendet**

1. Typ 1: Stop-And-Wait ARQ

- Nach jedem Paket wartet der Sender auf ein ACK
- Nur **ein Paket gleichzeitig in der „Luft“**
- **Einfach**, aber **sehr ineffizient bei langen Leitungen** (hohe RTT)
- Siehe auch: **Alternating Bit Protocol (ABP)**

2. Typ 2: Go-Back-N ARQ

- Sender kann **mehrere Pakete auf einmal** losschicken (Fensterstechnik)
- Wenn ein Paket verloren geht oder fehlerhaft ist:
 - Empfänger **verwirft alle folgenden Pakete**, da sie „out of order“ sind
 - Sender wird per Timeout oder NACK benachrichtigt und **sendet alle ab dem fehlerhaften Paket erneut**
- **Warum?**
 - Früher hatten viele Empfänger nur **einen Pufferplatz** → konnten keine out-of-order Pakete speichern

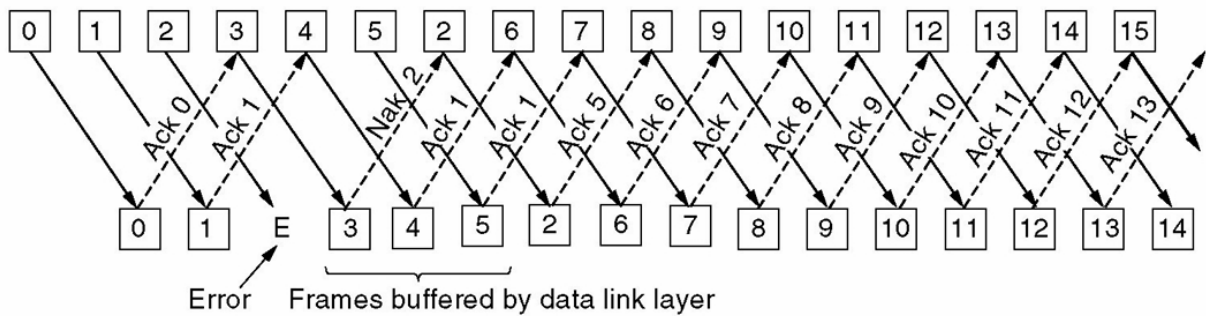


3. Typ 3: Selective Reject (Selective Repeat)

- **Effizienter:**

- Empfänger kann mehrere Pakete puffern
- Nur das **konkrete verlorene Paket** wird erneut angefordert
- Sender muss gezielt **nur das fehlende Paket** erneut senden
- Vorteil:

* **Weniger Overhead, bessere Bandbreitenausnutzung**



Intermezzo: Wie macht es TCP?

- TCP arbeitet **nicht mit Paketnummern**, sondern mit **Bytenummern**
 - Beispiel: Byte 513, Byte 769, ...
- Das erlaubt eine **feinere Steuerung** und **Teilmengenbestätigung**
- ABER: Moderne TCP-Implementierungen sind **komplex**, da sie **Abwärtskompatibilität** wahren müssen
 - Deshalb kombinieren sie **ACKs mit Bytenummern**, ggf. mit **SACK (Selective ACKs)** für selektive Wiederholungen

4.4 Congestion Control

Grundidee: Wo entsteht Überlast (Congestion)?

Bei jeder Kommunikation im Internet sind **mehrere Komponenten beteiligt**:

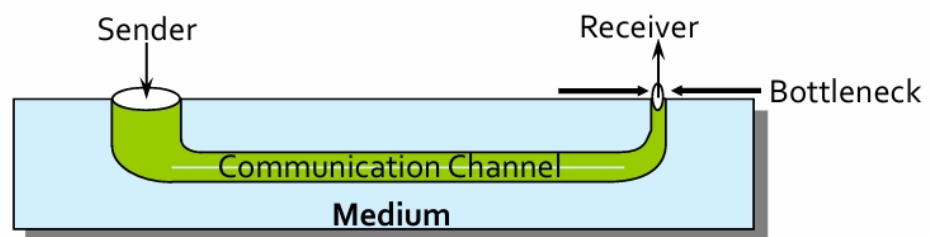
1. **Sender (Initiator)**
2. **Netzwerk (Router, Switches, Leitungen etc.)**
3. **Empfänger (Responder)**

Überall dort kann ein **Engpass (Bottleneck)** entstehen

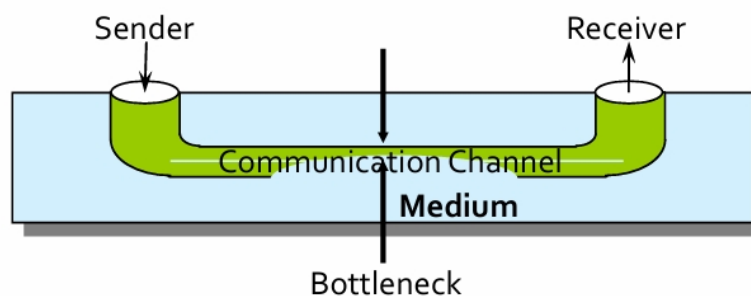
Drei mögliche Bottlenecks:

Am Empfänger	Receiver kann nicht schnell genug lesen oder verarbeiten
Im Kommunikationskanal	Leitung oder Router sind überlast → Netzüberlastung entsteht
Am Sender (selten)	Sender limitiert seine Rate selbst, z.B. bei Bandbreitenbegrenzung

Bottleneck at the Receiver

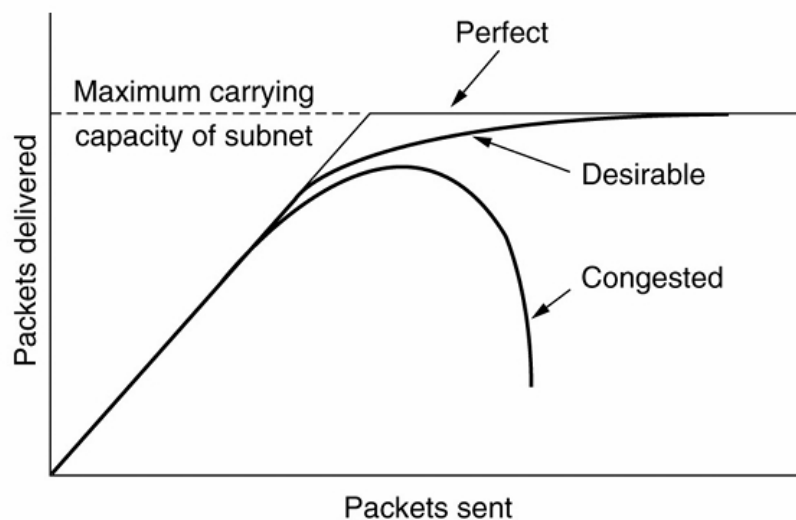
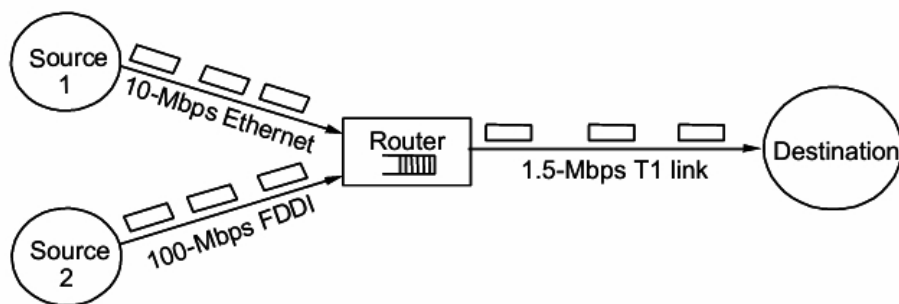


Bottleneck in the Communication Channel



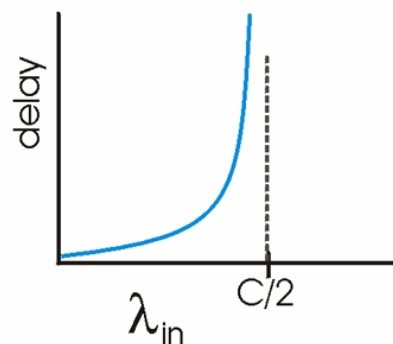
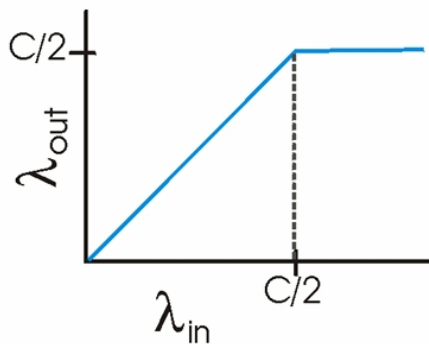
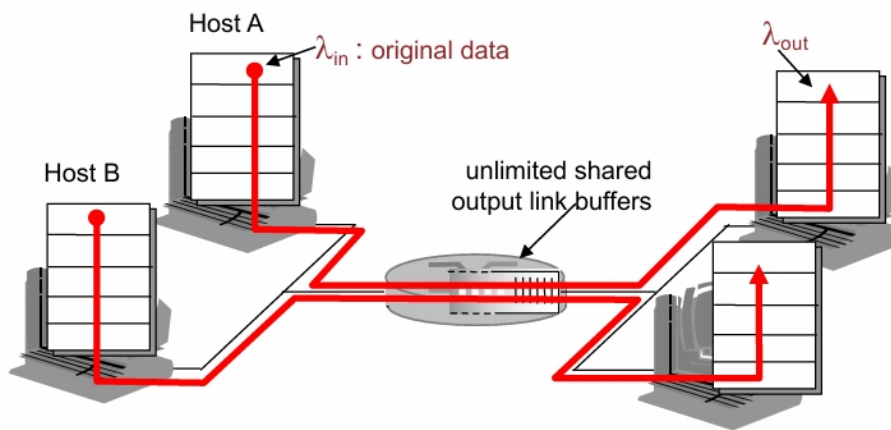
Was ist Congestion (Überlast)?

- **Jedes Netzwerk** hat eine **begrenzte Kapazität** (Daten pro Sekunde)
- Wenn **mehr Daten eingespeist** werden, als das Netzwerk verarbeiten kann, entsteht eine **Überlastung**
- Folge:
 - **Verzögerungen** (Delays)
 - **Paketverluste** (Drops)
 - **Ineffizienz**, da gleiche Pakete oft mehrmals übertragen werden müssen
 - **Congestive Collapse**: Netzwerk ist aktiv, aber es kommt kaum Daten durch → total ineffizient



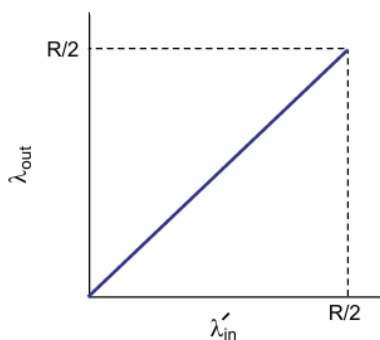
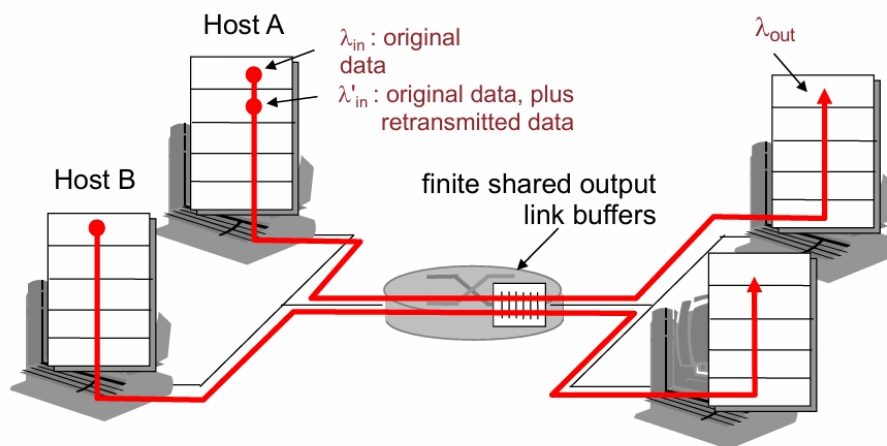
Drei Szenarien zur Veranschaulichung

- **Szenario 1 - Router mit unendlichem Puffer, keine Retransmission**
 - Zwei Sender und zwei Empfänger kommunizieren über einen Router
 - Der Router hat **unendlich große Puffer**, sodass **keine Pakete verloren gehen**
 - Die **eingehende Rate** λ_{in} kann die **Ausgangskapazität** C des Routers überschreiten
 - Pakete stauen sich im Puffer, was zu **wachsenden Verzögerungen** führt
 - Es entsteht **Congestion durch Verzögerung**, nicht durch Verluste
 - Das Ergebnis: **Maximale Dursatzrate** C wird erreicht, aber mit großen Latenzen

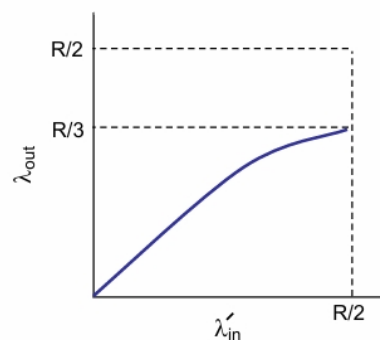


• **Szenario 2 - Router mit begrenztem Puffer, mit Retransmission**

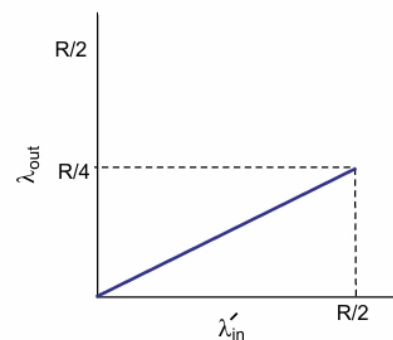
- Router hat **begrenzten Pufferplatz**; bei Überlast werden Pakete **verloren**
- Sender erkennen verlorene Pakete (z.B. durch Timeout) und **senden diese erneut** (Retransmission)
- Die Wiederholungen erhöhen die eingehende Daterate λ_{in} zusätzlich
- Dadurch entsteht ein **Teufelskreis: mehr Last → mehr Verluste → mehr Retransmissionen → noch mehr Last**
- Ergebnis: **hohe Verzögerungen, verminderte Effizienz**, und Gefahr von **Congestion Collapse** trotz physikalischer Kapazitätsgrenze C



Send only when „room“ in router



Retransmit only lost packets



Retransmit after timeout

• **Szenario 3: Mehrere Sender, Mehrhops und Retransmissionen**

1. **Ausgangslage:**

- Mehrere Sender (z.B. 4 Hosts) senden gleichzeitig
- Daten passieren mehrere Router (Multihop)
- Router haben begrenzte Pufferkapazität

2. **Problem:**

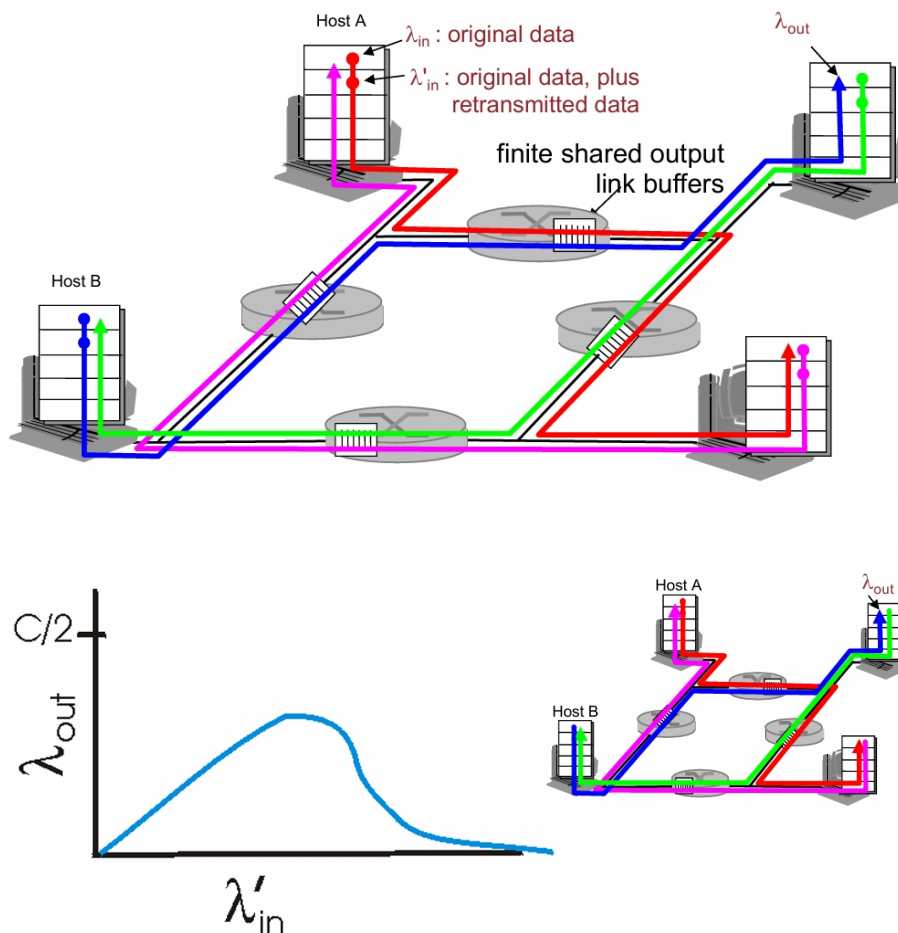
- Pakete gehen an Routern verloren, wenn Puffer voll sind
- Sender erkennen Verluste erst nach Timeout und senden Pakete erneut (Retransmission)

3. **Folge:**

- Retransmissionen erhöhen den Datenverkehr zusätzlich
- Mehr Last führt zu noch mehr Paketverlusten und Verzögerungen
- Ressourcen (Bandbreite und Puffer) werden verschwendet
- Gefahr von Congestion Collapse steigt

4. **Herausforderung:**

- Staukontrolle ist schwierig, weil Überlast an verschiedenen Punkte entsteht
- Kein garantierter Paketreihenfolge oder direkte Verlustmeldung



- **Warum ist Congestion Control nötig?**
 - Verhindert Überlastungs-Kettenreaktionen (Snowball-Effekt)
 - Überlast führt zu Paketverlusten → erneute Sendungen → noch mehr Last → Zusammenbruch
- **Anpassung der Sendrate:**
 - Sender passen ihre Rate an die aktuelle Kapazität des Netzwerks an
 - Komplex, da viele Router, Weiterleitungsstrategien und Nutzer beteiligt sind
 - Unterschied zu Flow Control: Flow Control ist lokal (Sender-Empfänger), Congestion Control betrifft das gesamte Netzwerk dazwischen
- **Ziele der Congestion Control:**
 - Hohe Paketlieferung mit kurzen Verzögerungen
 - Schutz vor kollapsartiger Überlast
 - Faire Verteilung der Netzwerkressourcen auf alle Datenflüsse
- **Designoptionen:**
 - **Open Loop:** Vorab so gestaltet, dass Laufzeitkorrekturen nicht nötig sind
 - **Closed Loop:** Laufzeit-Feedback zur Anpassung der Sendrate
 - * **Explizites Feedback:** Netzwerk signalisiert Überlast direkt
 - * **Impliziertes Feedback:** Sender schließt aus Netzwerkverhalten (z.B. fehlende ACKs) auf Überlast
- **Beispiel:** TCP verwendet implizites Closed-Loop Feedback zur Steuerung der Sendrate

Mögliche Maßnahmen bei Überlastung

- **Optionen zur Überlastvermeidung**
 1. **Kapazität erhöhen**
 - Mehr Links oder Router hinzufügen
 - **Aber:** In der Praxis kurzfristig kaum machbar - teuer und langsam
 2. **Reservierung & Admission Control**
 - Neue Datenflüsse nur zulassen, wenn noch genügend Kapazität vorhanden ist
 - **Gilt vor allem für Leitungsvermittlung** (wie bei alten Telefonsystemen)
 - Funktioniert schlecht im Internet, da Informationen über den aktuellen Zustand selten verfügbar sind
 3. **Last reduzieren (feingranular)**
 - Quellen verringern ihre Datenrate, **ohne** die Verbindung zu beenden
 - Das funktioniert **nur**, wenn sie **Feedback** vom Netzwerk bekommen - z.B. durch verlorene Pakete → Closed-loop control

Klassifikation von Maßnahmen

- **Wo wird gesteuert?**
 - **Router-zentriert:** Router trifft Entscheidungen
 - **Host-zentriert:** Endgeräte (Sender/Empfänger) treffen Entscheidungen
 - * Im Internet fast immer **host-zentriert, außer beim Packet-Dropping**
- **Wie wird die zulässige Datenmenge beschrieben?**
 - **Window-basiert (wie TCP):**
 - * Der Sender darf nur ein bestimmtes Fenster an Bytes senden, ohne ACK
 - **Rate-basiert:**
 - * Eine feste Datenrate (z.B. 100KB/s)
 - Weitere, seltenere Methoden: z.B. **Credit-based**

Was tun, wenn der Router voll ist?

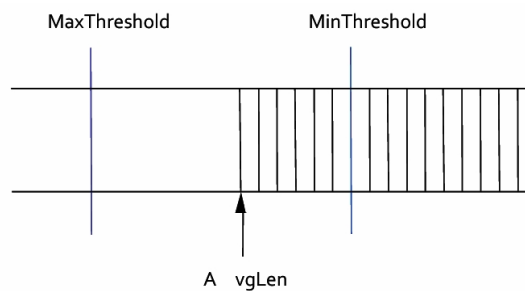
- **Ein neues Paket kommt an, aber der Puffer ist voll**
 - Muss **ein Paket gedroppt werden** - welches?
 - * **Option 1: Drop Tail**
 - Das neue Paket wird einfach verworfen
 - Vorteil: alte Pakete sind oft „wertvoller“ (z.B. bei Go-Back-N)
 - **Nachteil:** Gefahr der Synchronisation - alle Sender reduzieren gleichzeitig
 - * **Option 2: Ein altes Paket verwerfen**
 - Besonders sinnvoll bei **Multimedia-Streams** → neue Daten wichtiger
 - Oder gezielt ein Paket aus dem gleichen Datenstrom löschen
 - **Aber:** schwer umzusetzen (Overhead)

Paketverlust = implizites Feedback

- **Paketverlust als Signal**
 - Wenn ein Paket **verloren geht**, kann der Sender (z.B. TCP) das durch fehlendes ACK erkennen
 - Reaktion: **Sende-Rate verringern**
 - **Annahme dabei:** Verluste = Überlast (gilt in drahtgebundenen Netzen oft, aber nicht immer in drahtlosen)
- **Problem:**
 - In drahtlosen Netzwerken: viele Verluste **ohne Überlast**, z.B. durch Interferenzen
 - Lösung: Drahtlose Hops absichern (z.B. durch Link-Layer-Retransmissions)
- **Open-loop Control:**
 - Falls trotz Ressourcenreservierung Pakete an vollen Queues ankommen → schlechtes Design

Proaktive Maßnahmen zur Vermeidung voller Queues

- **Idee:**
 - Nicht erst reagieren, **wenn die Queue voll ist**, sondern schon **vorher!**
- **Router erkennt Überlast frühzeitig, z.B.:**
 - Durchschnittliche Queue-Länge über Grenzwert
 - Auslastung des Ausgangslinks konstant hoch
 - → Router wechselt in den **Warnzustand**



Proaktive Maßnahme: Choke Packets

- **Choke Packet**
 - Ein spezielles Paket, das zurück zum Sender geschickt wird: „Langsamer senden!“
- **Probleme:**
 1. Führt zu **noch mehr Datenverkehr**, obwohl das Netz schon überlastet ist
 2. **Verzögerung:** Bis der Sender das Feedback erhält, hat er oft schon viele Pakete gesendet (Stichwort: **Bandwidth-Delay Product**)

Proaktive Maßnahme: Warning Bit

- **Alternative zu Choke Packets:**
 - Router setzt ein **Warnbit** im Header der Datenpakete
 - Der Empfänger kopiert das Bit in die **ACKs**
 - Der Sender sieht das Warnbit in den ACKs → verringert seine Sende-Rate
 - Vorteil: Kein zusätzliches Paket nötig - **effizienter**

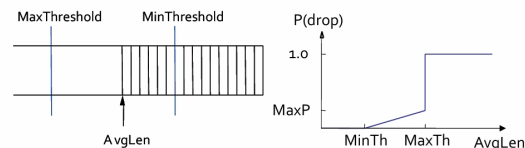
Proaktive Maßnahme: Random Early Detection (RED)

- **RED-Mechanismus**

- Wenn sich Überlast **anbahnt**, werden **einige Pakete frühzeitig absichtlich verworfen!**
- Klingt brutal, aber:
 - * Führt dazu, dass Sender **frühzeitig drosseln**
 - * So wird verhindert, dass **alle Pakete** verloren gehen, wenn es richtig voll wird

- **Dropprobabilität**

- Beginnt bei 0 (unter **MinThreshold**)
- Steigt **linear** zwischen MinThreshold und MaxThreshold
- Ab MaxThreshold: **100% Dropprobabilität**



Reaktion auf Feedback

Was macht der Sender, wenn er Feedback bekommt?

- **Rate-basierte Protokolle:**

- Verringern einfach die Sende-Rate (z.B. um Faktor 2)
- Frage: Wie und wann wieder erhöhen?

- **Window-basierte Protokolle (wie TCP):**

- Verringern **Congestion Window**
- Viele Strategien: z.B. „Multiplicative Decrease, Additive Increase“
- Frage: Wie schnell darf das Fenster wachsen?
- Große → Burst-Sendungen → **kann zu neuer Überlast führen**

4.5 Internet UDP

Warum gibt es UDP überhaupt?

1. **Kein Verbindungsaufbau = weniger Verzögerung:**

- TCP z.B. muss man erst eine Verbindung aufbauen, was Zeit kostet
- UDP sendet einfach drauflos

2. **Einfachheit:**

- Kein Verbindungsstatus wird am Sender oder Empfänger gespeichert
- Keine aufwändige Steuerung

3. **Kleiner Header:**

- UDP-Header ist nur 8 Bytes lang (im Gegensatz zu TCP mit 20 Bytes oder mehr)

4. **Keine Staukontrolle (Congestion Control)**

- UDP schickt Pakete so schnell wie möglich
- Kein langsamer machen bei Netzwerküberlastung

5. **Warum nicht nur IP?**

- IP liefert Pakete nur bis zum Zielhost
- UDP ermöglicht mehrere Anwendungen auf demselben Host (über Ports)
- UDP ergänzt IP um eine einfache Multiplexing-Funktion (Ports)

Wofür wird UDP oft verwendet?

• **Streaming Multimedia:**

- Video, Audio, Live-Übertragungen
- Verlust einiger Pakete ist tolerierbar
- Wichtig ist, dass Daten schnell ankommen, nicht unbedingt komplett oder in der richtigen Reihenfolge

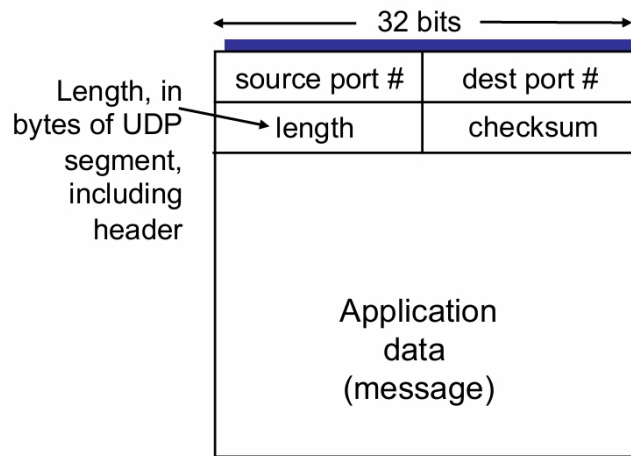
• **Andere Beispiele:**

- **DNS** (Domain Name System)
- **SNMP** (Netzwerkmanagement)
- Anwendungen, die ihre eigene Fehlerbehandlung machen

Zuverlässigkeit & Reihenfolge

- UDP garantiert **keine Reihenfolge** und keine Zuverlässigkeit
- Wenn Anwendungen zuverlässige oder geordnete Daten wollen, müssen sie selbst **Sequenznummern und Fehlerkorrektur** einbauen
- Beispiel: TCP ist zuverlässig, UDP nicht - aber man kann mit UDP selbst sowas programmieren, wenn man will

UDP-Segment Aufbau (Header)

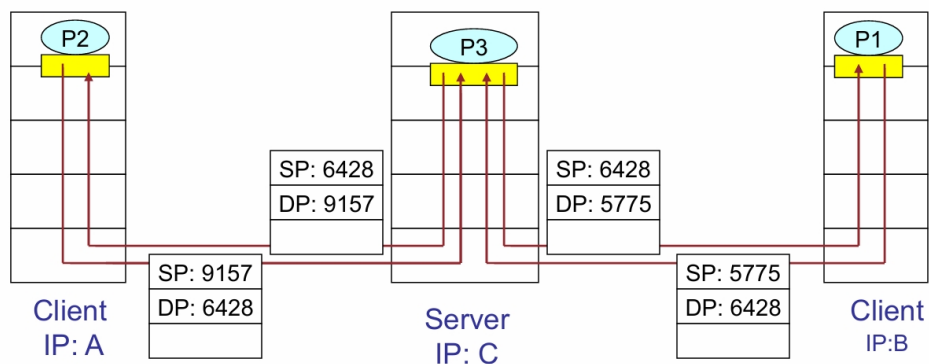


UDP segment format

Verbindungslos und Demultiplexing

- UDP benutzt **Portnummern**, um Pakete an die richtige Anwendung weiterzuleiten (Demultiplexing)
- Beispiel: Ein Rechner kann mehrere UDP-Sockets öffnen, z.B. auf Port 64285, 99111, 99222
- Wenn ein UDP-Paket ankommt, schaut das Betriebssystem auf die **Zielporennummer** und gibt das Paket an die passende Anwendung (Socket) weiter
- Da UDP **verbindungslos** ist, gibt es keinen dauerhaften Kanal. Jedes Paket wird einzeln verarbeitet

```
DatagramSocket serverSocket = new DatagramSocket(6428);
```



Source port (SP) provides "return address"

Wichtige Punkte

- **Socket = IP-Adresse + Port-Nummer** (zweier Tupel, nicht nur Port)
- Mehrere Pakete von verschiedenen Quellen mit verschiedenen Quell-IP und Quell-Port können an denselben Ziel-Port (Socket) gehen
- Der Quellport dient als „**Rückadresse**“, damit Antworten zum richtigen Sender zurückgehen

Ziel der UDP-Checksumme

- **Fehlererkennung:** Prüfen, ob beim Übertragen eines Segments (UDP-Datenpaket) z.B. **Bits gekippt** sind (z.B. $0 \rightarrow 1$ oder $1 \rightarrow 0$)
- Die UDP-Checksumme ist **einfach**, aber effektiv gegen viele Übertragungsfehler

Wie funktioniert die UDP-Checksumme?

- **Sender-Seite (vor dem Versand):**
 1. **Teile das gesamte UDP-Segment** (Header + Daten) in **16-Bit-Wörter** auf
 2. **Addiere alle 16-Bit-Wörter** miteinander
 - Bei Überlauf (also „Carry out“ von links), **wrap-around** → du addierst ihn **wieder dazu**
 3. **Nimm das 1-Komplement** (also invertiere jedes Bit: $0 \rightarrow 1$, $1 \rightarrow 0$)
 4. **Schreibe dieses Ergebnis in das Checksum-Feld** des UDP-Headers
 5. Versende das Segment
- **Empfänger-Seite (nach Empfang):**
 1. Erhalte das UDP-Segment inkl. Checksum-Wert
 2. **Addiere alle 16-Bit-Wörter** (inkl. der empfangenen Checksum)
 3. Auch hier: Bei Überlauf - Wrap-around wieder addieren
 4. **Ergebnis sollte 0xFFFF sein** (alle Bits = 1)
 - Wenn ja → vermutlich **kein Fehler**
 - Wenn nein → **Fehler erkannt**, Segment wird verworfen

Beispiel: Internet Checksum

	1	1	1	0	0	1	1	0	0	1	1	0	0	1	1	0
	1	1	0	1	0	1	0	1	0	1	0	1	0	1	0	1
wraparound	1	0	1	1	1	0	1	1	1	0	1	1	1	0	1	1
sum	1	0	1	1	1	0	1	1	1	0	1	1	1	1	0	0
checksum	0	1	0	0	0	1	0	0	0	1	0	0	0	0	1	1